

JAVA SDE-2 INTERVIEW PREPARATION SERIES

DATA STRUCTURES AND ALGORITHMS - BOOK 15 OF 17 - STUDY STEP DSA-15

Graphs: Traversal, Ordering, Paths, and Union-Find

Modeling, BFS, DFS, Topological Sort, Shortest Paths, and Connectivity

BY

Vinay Reddy Kalluri

Translate domains into vertices and edges, then choose traversal, ordering, path, or connectivity algorithms from graph constraints.

MODELING | BFS | DFS | TOPOLOGICAL SORT | DIJKSTRA | UNION-FIND

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

60-second entry rule: If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to [DSA 14 - Heaps and Priority Queues](#). If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Graph problems become manageable once the model and edge semantics are explicit. This volume emphasizes algorithm selection, state, overflow, and correctness.

Readiness map

Decision	Evidence
START HERE	Entities are connected by arbitrary relationships; Dependencies require an order or cycle test; Reachability, components, or shortest paths are requested; Grid cells can be modeled as graph vertices.
PREPARE FIRST	JAVA-01 and DSA-01 through DSA-14; Queues, recursion, heaps, and hashing.
MOVE ON WHEN	Model graph state deliberately; Implement BFS, DFS, topological sort, Dijkstra, and union-find; Select a shortest-path algorithm from weight constraints; Explain complexity using V and E.

Choose the route that fits today

- 1 **Learning:** read in order, type the representative Java examples, and complete each dry run.
- 2 **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

DSA Segment Position

DSA 14 - Heaps and Priority Queues	YOU ARE HERE DSA 15 - Graphs	DSA 16 - Greedy Algorithms
------------------------------------	---------------------------------	----------------------------

A note from Vinay

Before choosing a graph algorithm, write down direction, weight rules, connectivity, and the requested output. Many advanced-looking failures begin with solving the wrong graph model perfectly.

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION	
Start Here - Your Route Through This Volume	2
Part I - Core Learning	4
Chapter 1 - Graph Foundations: Model the Relationships Before Traversal	4
Chapter 2 - SDE-2 Graph Modeling and Algorithm Patterns	8
Chapter 3 - Essential Graph Frontier and Weight Clinics	33
Chapter 4 - Realistic Graph Interview Rounds	37
Chapter 5 - Advanced Graph Algorithms, Implemented and Defended	41
Chapter 6 - Union-Find and Incremental Connectivity	53
Chapter 7 - Implicit Graphs and State-Space Search	63
Chapter 8 - Graph Practice Lab	72
Chapter 9 - Graph Practice Lab Solutions	74
Chapter 10 - Executable Graph Algorithm Checks	76
Part II - Practice and Handoff	92

Part I - Core Learning**SDE-2 CORE CHAPTER**

Chapter 1 - Graph Foundations: Model the Relationships Before Traversal

A graph contains vertices and edges. The hardest early step is often deciding what each represents.

- Social network: person is a vertex, friendship is an undirected edge.
- Build system: task is a vertex, prerequisite is a directed edge.
- Road network: location is a vertex, road is a weighted edge.
- Grid: valid cell is a vertex, allowed move is an implicit edge.

Vocabulary that changes the algorithm

Property	Meaning	Consequence
Directed	$u \rightarrow v$ need not imply $v \rightarrow u$	cycle and reachability logic changes
Undirected	edge connects both ways	parent edge must not look like a cycle
Weighted	edges carry cost	BFS is insufficient for arbitrary weights
Connected	every vertex reaches every other in undirected graph	otherwise loop over components
DAG	directed and acyclic	topological order exists

A path is a sequence of adjacent vertices. A cycle returns to an earlier vertex. A component is a maximal connected region under the relevant direction semantics.

Adjacency-list representation

For vertices numbered $0 \dots n-1$:

JAVA

```
static List<List<Integer>> undirectedGraph(int vertices, int[][] edges) {
    List<List<Integer>> graph = new ArrayList<>(vertices);
    for (int i = 0; i < vertices; i++) graph.add(new ArrayList<>());
    for (int[] edge : edges) {
        int from = edge[0];
        int to = edge[1];
        graph.get(from).add(to);
        graph.get(to).add(from);
    }
    return graph;
}
```

An adjacency list uses $O(V + E)$ space for a directed graph and stores two adjacency entries per undirected edge. An adjacency matrix uses $O(V \text{ squared})$ space and provides $O(1)$ edge-existence checks.

Validate vertex ranges and clarify parallel edges and self-loops. Those details affect indegrees, cycle rules, and costs.

BFS from zero

Breadth-first search explores vertices in increasing unweighted edge distance from a source.

JAVA

```
static int[] distances(List<List<Integer>> graph, int source) {
    int[] distance = new int[graph.size()];
    Arrays.fill(distance, -1);
    Deque<Integer> queue = new ArrayDeque<>();
    distance[source] = 0;
    queue.addLast(source);
    while (!queue.isEmpty()) {
        int node = queue.removeFirst();
        for (int neighbor : graph.get(node)) {
            if (distance[neighbor] == -1) {
                distance[neighbor] = distance[node] + 1;
                queue.addLast(neighbor);
            }
        }
    }
    return distance;
}
```

Setting distance when enqueueing also marks visited. Each vertex enters the queue once. For unweighted graphs, the first discovery is along a shortest path.

DFS from zero

Depth-first search completes one branch before trying the next. It supports reachability, components, cycle state, entry/exit timing, and postorder.

JAVA

```
static void dfs(List<List<Integer>> graph, int node, boolean[] visited) {
    visited[node] = true;
    for (int neighbor : graph.get(node)) {
        if (!visited[neighbor]) dfs(graph, neighbor, visited);
    }
}
```

Recursive depth can reach $O(V)$, unsafe for a deep graph in Java. An explicit [ArrayDeque](#) avoids call-stack failure.

Disconnected graphs

One traversal from vertex 0 sees only its reachable component. To count components:

JAVA

```
for each vertex:
    if unseen:
        components++
        traverse from it
```

This outer loop is not extra $O(VE)$ work. Across all traversals, each vertex and adjacency entry is processed a bounded number of times: $O(V + E)$.

Grid as an implicit graph

Do not allocate adjacency lists for a grid unless needed. Generate up/down/left/right neighbors from coordinates. Decide whether diagonal movement, wrapping, blocked cells, and jagged rows are valid.

Use a direction table:

JAVA

```
int[][] directions = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
```

Mark before enqueue/recurse to prevent duplicate work. Mutating the grid as visited saves a separate array only when the input-mutation contract allows it.

Choose shortest-path logic from weights

Edge weights	Typical algorithm
all equal/unweighted	BFS
only 0 and 1	0-1 BFS with deque
nonnegative	Dijkstra with min-heap
negative edges, no reachable negative cycle	Bellman-Ford
all-pairs, dense/moderate V	Floyd-Warshall
DAG	topological relaxation

Dijkstra is not correct with negative edges. A visited set alone is not enough to choose an algorithm; weight constraints drive the choice.

Topological ordering

In a directed graph, an edge $u \rightarrow v$ can mean u must happen before v . Kahn's algorithm queues all zero-indegree vertices, removes them, and decrements neighbors. If fewer than V vertices are emitted, a directed cycle prevents a complete order.

Union-Find boundary

Disjoint-set union maintains evolving undirected connectivity under edge additions. It does not provide paths and does not directly handle deletions. Path compression and union by rank/size give near-constant amortized operations.

Complexity language

For adjacency lists, traversal is $O(V + E)$, not simply $O(n)$. Memory includes the graph representation plus $O(V)$ visited/frontier state. For a grid with rows and columns, V is valid cells and E is proportional to allowed neighbor relationships.

TRY IT | Foundation checkpoint

- 1 What are vertices and edges in the problem domain?
- 2 Is the graph directed, weighted, disconnected, or implicit?
- 3 Why should BFS mark at enqueue time?
- 4 Why does one DFS not necessarily visit every vertex?
- 5 Which weight contract makes Dijkstra valid?

SDE-2 CORE CHAPTER

Chapter 2 - SDE-2 Graph Modeling and Algorithm Patterns

Why graph interviews begin before traversal

A graph is a model of entities and relationships. The same business data can become directed or undirected, weighted or unweighted, static or streaming, simple or multi-edge. That choice determines which algorithm is correct. Running Dijkstra on a graph that contains negative edges is not a small implementation bug; it is a modeling and precondition failure.

At SDE-2 level, begin by defining vertices, edges, direction, weight meaning, duplicate/self-loop policy, and scale. Then choose a representation and algorithm. A complete answer states the invariant that makes a traversal, relaxation, or cut decision safe, reconstructs requested outputs, and identifies the boundary where an in-memory textbook solution no longer fits.

Learning objectives

After completing this chapter, you should be able to:

- translate a domain problem into a graph with explicit direction and weight semantics;
- compare adjacency lists, matrices, edge lists, and implicit graphs;
- implement BFS, DFS, connected components, and grid BFS;
- detect undirected and directed cycles and test bipartiteness;
- derive topological sorting from indegree or DFS finishing state;
- choose among BFS, Dijkstra, Bellman-Ford, DAG relaxation, and Floyd-Warshall;
- run Dijkstra with stale-entry handling and reconstruct a shortest path;
- derive minimum spanning trees with Kruskal/DSU and explain Prim's alternative;
- identify SCC and bridge algorithms as advanced graph-decomposition tools; and
- discuss stack depth, memory, disconnected data, large graphs, and distributed boundaries.

Model the graph explicitly

Before selecting an algorithm, answer:

- 1 **Vertex:** what is one node, and how is it identified?
- 2 **Edge:** what relation creates an edge?
- 3 **Direction:** does $u \rightarrow v$ imply $v \rightarrow u$?
- 4 **Weight:** cost, time, distance, probability, capacity, or something else?
- 5 **Multiplicity:** can parallel edges or self-loops exist?
- 6 **Dynamics:** is the graph fixed during the query?
- 7 **Output:** reachability, one path, all distances, an ordering, components, or a spanning structure?
- 8 **Scale:** how many vertices and edges, and can they fit in memory?

Examples:

- Build prerequisites are directed edges from prerequisite to dependent task.
- Mutual friendship is usually undirected; following is directed.
- A grid is an implicit graph whose open cells are vertices and legal moves are edges.
- Currency exchange with multiplicative rates needs a transformed model, not an unexamined "weighted shortest path."
- Network capacity is not a shortest-path weight; it may require max flow.

If a problem says "least number of transfers," every transition has unit cost and BFS is the first choice. If it says "least travel time" with nonnegative times, use Dijkstra. Meaning precedes syntax.

Representations and cost

Let V be vertices and E edges.

Representation	Space	Enumerate neighbors	Test one edge	Best fit
adjacency list	$O(V + E)$	$O(\text{degree}(v))$	usually $O(\text{degree}(v))$	sparse graphs and traversal
adjacency matrix	$O(V^2)$	$O(V)$	$O(1)$	dense small graphs
edge list	$O(E)$	$O(E)$ without index	$O(E)$	Kruskal, Bellman-Ford, interchange
implicit neighbors	domain state only	cost to generate	domain-specific	grids, puzzles, generated states

For weighted Java graphs, an adjacency list may be `List<List<Edge>>`. For dense integer IDs, `int[][]` or primitive arrays reduce boxing. For external string IDs, map them to compact integer indexes and retain a reverse mapping for output.

An undirected edge must appear in both adjacency lists unless the traversal generates both directions. Parallel edges may be valid; deduplicating them can change path or capacity semantics. Self-loops immediately form directed cycles and undirected cycles under many definitions, so state the chosen graph model.

WHY IT WORKS | Universal traversal invariant

BFS and DFS differ in frontier order, not reachability correctness. Both maintain:

- discovered vertices have been scheduled at most once;
- every scheduled vertex is reachable from the start through discovered edges; and
- when a vertex is processed, all its outgoing edges are examined.

Mark a vertex discovered when adding it to the frontier, not when removing it. Marking late permits many duplicate enqueues on converging edges and can destroy linear bounds.

Traversal time is $O(V + E)$ for an adjacency list because each vertex is scheduled once and every adjacency entry is examined once. For an undirected list, each logical edge appears twice, which remains $O(E)$.

Pattern 1: breadth-first search

BFS uses a FIFO queue. It processes vertices by nondecreasing number of edges from the source. In an unweighted graph, the first discovery of a vertex therefore gives a shortest edge-count path.

Invariant when dequeuing a vertex at level d : every vertex at a smaller distance has been processed, and no undiscovered path with fewer than $d + 1$ edges remains. Assign a neighbor distance $d + 1$ when it is first discovered and optionally store its parent.

TRACE IT | Dry run

For adjacency $0: [1, 2], 1: [0, 3], 2: [0, 3], 3: [1, 2, 4], 4: [3]$, BFS from 0 visits 0, 1, 2, 3, 4. Levels are 0, 1, 1, 2, 3. Either 0-1-3-4 or 0-2-3-4 is a valid shortest path, depending on neighbor order.

BFS memory can be $O(V)$ and its frontier may be very wide. Bidirectional BFS can reduce explored depth for one source-target query in an unweighted graph when reverse neighbors are available and branching is high.

Pattern 2: depth-first search

DFS explores one branch before backtracking. It supports reachability, components, cycle state, topological finishing order, and low-link decompositions.

Recursive DFS mirrors the proof but can overflow the Java stack on a long chain. An explicit `ArrayDeque<Integer>` is safer for untrusted or large graphs. Pushing neighbors in reverse gives a deterministic left-neighbor preference, although mark-on-push behavior around cross edges can still differ from recursive visitation order.

Invariant: the stack contains discovered work whose reachable outgoing edges are not all processed. Popping a vertex in the simple iterative traversal commits it to output; every newly pushed neighbor has a discovered path through the current vertex.

DFS remains $O(V + E)$ and may hold $O(V)$ frontier/visited state. Do not claim $O(\text{depth})$ total space while ignoring the visited set required for a general cyclic graph.

Pattern 3: connected components

For an undirected graph, scan every vertex. When a vertex is unseen, increment the component count and traverse from it. That traversal marks exactly one connected component.

Invariant before scan index v : every earlier vertex belongs to a fully marked component. An unseen v cannot belong to any already traversed component, or reachability would have marked it. It therefore starts a new component.

This handles isolated vertices, which count as size-one components. Time is $O(V + E)$, space $O(V)$. A disjoint-set union structure is an alternative when edges arrive incrementally and queries ask whether vertices are currently connected.

Pattern 4: grid as an implicit graph

For four-direction movement, each open cell has up to four neighbors. BFS finds the fewest steps when every move costs one. Validate rectangularity, coordinates, and blocked endpoints.

Use a distance matrix initialized to -1 . Mark the start distance zero on enqueue. For each dequeued cell, generate in-bounds open neighbors whose distance is still -1 .

For the grid below, S is the start, T is the target, and $\#$ blocks movement:

TEXT

```
S..
.#.
..T
```

One shortest route moves right, right, down, down for four steps. Time and memory are $O(\text{rows} * \text{cols})$. If moves have weights 0 and 1, use 0-1 BFS with a deque; for arbitrary nonnegative costs, use Dijkstra.

Pattern 5: undirected cycle detection

During DFS of an undirected graph, an edge back to the immediate parent is the same tree edge viewed in reverse and is not a cycle. An edge to any other discovered vertex proves a cycle.

Carry `(vertex, parent)` state. Invariant: the DFS tree edges connect each discovered nonroot vertex to its parent. Encountering a previously discovered neighbor different from that parent closes a path through the DFS tree.

Parallel undirected edges complicate a parent-by-vertex check: the second parallel edge to the parent can itself form a length-two multi-edge cycle. If multigraphs are allowed, assign edge IDs and skip only the exact incoming edge ID. The runnable helper assumes a simple symmetric graph.

Pattern 6: directed cycles and topological order

Directed DFS uses three colors:

- white: undiscovered;
- gray: active on the current recursion path;
- black: fully processed.

An edge to gray is a back edge and proves a directed cycle. An edge to black does not, because that vertex's path has already finished.

Kahn's topological algorithm gives an iterative alternative. Compute indegrees, enqueue every zero-indegree vertex, repeatedly emit one and decrement its outgoing neighbors. Invariant: every queued vertex has no incoming edge from the remaining graph, so placing it next respects all dependencies. If fewer than V vertices are emitted, the remaining subgraph has no zero-indegree vertex and contains a cycle.

A topological order exists only for a directed acyclic graph and may not be unique. Use a priority queue instead of a FIFO queue if the smallest deterministic vertex should be chosen at each step, increasing frontier operations to $O(\log V)$.

Pattern 7: bipartite testing

A graph is bipartite when vertices can be colored with two colors so every edge crosses colors. BFS or DFS each component, assigning an uncolored neighbor the opposite color. An edge whose endpoints have the same color proves failure.

Invariant: every processed edge among colored vertices crosses colors. Because disconnected graphs can contain the violating component, scan every vertex, not only one start.

Equivalently, an undirected graph is bipartite exactly when it has no odd-length cycle. A self-loop fails immediately. Time is $O(V + E)$, space $O(V)$.

Shortest-path decision table

Edge model	Algorithm	Typical time
unweighted/unit weight	BFS	$O(V + E)$
weights only 0 or 1	0-1 BFS	$O(V + E)$
nonnegative weights	Dijkstra with binary heap	$O((V + E) \log V)$
DAG with any weights	topological relaxation	$O(V + E)$
negative edges, detect reachable negative cycle	Bellman-Ford	$O(VE)$
all-pairs, dense and moderate V	Floyd-Warshall	$O(V^3)$ time, $O(V^2)$ space

A negative cycle reachable from the source makes shortest distance undefined for vertices reachable from that cycle: the path cost can decrease without bound. Bellman-Ford relaxes every edge $V - 1$ times, then a further successful relaxation detects such a cycle. Early termination is safe when one full pass makes no change.

Floyd-Warshall dynamic programming allows intermediate vertices one by one:

TEXT

```
dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
```

At phase k , the invariant is that distances use only intermediate vertices from the processed set. Guard unreachable sentinels before addition and choose a width that cannot overflow.

Pattern 8: Dijkstra with path reconstruction

Dijkstra maintains tentative distances and a min-priority queue. Repeatedly settle the reachable unsettled vertex with smallest distance, then relax its outgoing edges.

For edge $u \rightarrow v$ of weight w , relaxation tests whether $\text{dist}[u] + w < \text{dist}[v]$. If so, update distance, set $\text{parent}[v] = u$, and add a new queue state. Java's `PriorityQueue` has no efficient decrease-key, so old states remain. When polled, discard a state whose stored distance differs from the current $\text{dist}[\text{vertex}]$.

Correctness invariant: every settled vertex has its final shortest distance. Nonnegative weights are essential: any path reaching an unsettled vertex through another unsettled vertex cannot later reduce the current minimum by adding a negative amount.

TRACE IT | Dry run

Edges: $0 \rightarrow 1$ (4), $0 \rightarrow 2$ (1), $2 \rightarrow 1$ (2), $1 \rightarrow 3$ (1), $2 \rightarrow 3$ (5). Start 0.

- 1 Settle 0; tentative distances: $1=4$, $2=1$.
- 2 Settle 2; improve 1 to 3 through 2, set 3 to 6.
- 3 Settle 1 at 3; improve 3 to 4.
- 4 Settle 3 at 4. Old states for 1 at 4 and 3 at 6 are stale.

Reconstruct target 3 by parents $3 \leftarrow 1 \leftarrow 2 \leftarrow 0$, reverse to $[0, 2, 1, 3]$. If unreachable, return an explicit empty path with an infinite-distance sentinel rather than fabricating a route.

Minimum spanning tree versus shortest paths

For a connected undirected weighted graph, a minimum spanning tree (MST) connects every vertex with minimum total edge weight. It does not generally preserve shortest paths from any source.

Kruskal and DSU

Sort edges by increasing weight. Add an edge only when its endpoints currently belong to different components. A disjoint-set union (DSU) supports **find** with path compression and **union** by size/rank.

Cut property: for any cut of the vertices, a lightest edge crossing that cut is safe for some MST. Kruskal's next edge connects two current components and is a lightest remaining edge across their cut; adding it cannot create a cycle and is safe. Stop after $V - 1$ accepted edges.

Time is $O(E \log E)$ for sorting; DSU operations are nearly constant amortized, formally $O(\alpha(V))$. If fewer than $V - 1$ edges are accepted, the graph is disconnected. Return a minimum spanning forest only if that is the stated contract.

Prim

Prim grows one tree from a start vertex. A min-heap stores edges crossing from the included set to excluded vertices. Repeatedly choose the lightest crossing edge, skip it if its destination is already included, and add the new vertex's outgoing edges. The same cut property proves safety.

With adjacency lists and a binary heap, time is $O(E \log V)$. Prim is natural when the graph is already in adjacency form; Kruskal is natural for an edge list and gives explicit component merging.

Negative edge weights are allowed in MST algorithms. The nonnegative restriction belongs to Dijkstra, not to spanning trees.

Advanced decomposition boundary

Some SDE-2 interviews ask recognition and high-level mechanics rather than full code for these families.

- **Strongly connected components (SCC):** in a directed graph, every pair inside an SCC reaches each other. Kosaraju uses finishing order plus a reversed graph; Tarjan uses one DFS with discovery indexes, low links, and a stack. Condensing SCCs produces a DAG.
- **Bridges and articulation points:** in an undirected graph, a bridge removal increases component count; an articulation vertex removal does likewise. DFS low-link values track the earliest discovery reachable through tree edges plus at most one back edge. A tree edge (u, v) is a bridge when `low[v] > discovery[u]` in a simple graph.
- **Euler paths:** depend on vertex degrees and connectedness of nonzero-degree vertices, not on MST or shortest paths.
- **Network flow:** capacities and conservation require augmenting-path or blocking-flow algorithms, not greedy shortest paths alone.

Know the recognition signal, invariant vocabulary, complexity, and implementation hazards. Do not force every advanced graph problem into BFS, Dijkstra, or DSU.

Runnable Java 21 reference implementation

Run with `java -ea GraphInterviewPatterns`.

JAVA (continued 1/13)

```
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.PriorityQueue;

public final class GraphInterviewPatterns {
    private GraphInterviewPatterns() {}

    public record WeightedEdge(int to, int weight) {}

    public record UndirectedEdge(int first, int second, int weight) {}

    public record PathResult(long distance, List<Integer> path) {
        public boolean reachable() {
            return distance != Long.MAX_VALUE;
        }
    }

    private record NodeDistance(int vertex, long distance) {}

    public static List<Integer> bfsOrder(int[][] adjacency, int start) {
        validateAdjacency(adjacency);
        validateVertex(start, adjacency.length);
        boolean[] seen = new boolean[adjacency.length];
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        List<Integer> order = new ArrayList<>();
        seen[start] = true;
```

JAVA (continued 2/13)

```
        queue.addLast(start);
        while (!queue.isEmpty()) {
            int vertex = queue.removeFirst();
            order.add(vertex);
            for (int neighbor : adjacency[vertex]) {
                if (!seen[neighbor]) {
                    seen[neighbor] = true;
                    queue.addLast(neighbor);
                }
            }
        }
        return List.copyOf(order);
    }

    public static List<Integer> dfsOrder(int[][] adjacency, int start) {
        validateAdjacency(adjacency);
        validateVertex(start, adjacency.length);
        boolean[] seen = new boolean[adjacency.length];
        ArrayDeque<Integer> stack = new ArrayDeque<>();
        List<Integer> order = new ArrayList<>();
        seen[start] = true;
        stack.push(start);
        while (!stack.isEmpty()) {
            int vertex = stack.pop();
            order.add(vertex);
            int[] neighbors = adjacency[vertex];
            for (int i = neighbors.length - 1; i >= 0; i--) {
                int neighbor = neighbors[i];
                if (!seen[neighbor]) {
                    seen[neighbor] = true;
                    stack.push(neighbor);
                }
            }
        }
    }
}
```

JAVA (continued 3/13)

```
        return List.copyOf(order);
    }

    public static int countUndirectedComponents(int[][] adjacency) {
        validateAdjacency(adjacency);
        boolean[] seen = new boolean[adjacency.length];
        int components = 0;
        for (int start = 0; start < adjacency.length; start++) {
            if (seen[start]) {
                continue;
            }
            components++;
            ArrayDeque<Integer> stack = new ArrayDeque<>();
            seen[start] = true;
            stack.push(start);
            while (!stack.isEmpty()) {
                int vertex = stack.pop();
                for (int neighbor : adjacency[vertex]) {
                    if (!seen[neighbor]) {
                        seen[neighbor] = true;
                        stack.push(neighbor);
                    }
                }
            }
        }
        return components;
    }

    public static int shortestGridSteps(int[][] grid, int startRow, int startCol,
        int targetRow, int targetCol) {
        int cols = validateRectangularGrid(grid);
        validateCell(startRow, startCol, grid.length, cols);
        validateCell(targetRow, targetCol, grid.length, cols);
        if (grid[startRow][startCol] != 0 || grid[targetRow][targetCol] != 0) {
```

JAVA (continued 4/13)

```

        return -1;
    }
    int[][] distance = new int[grid.length][cols];
    for (int[] row : distance) {
        Arrays.fill(row, -1);
    }
    int[] rowDelta = {-1, 1, 0, 0};
    int[] colDelta = {0, 0, -1, 1};
    ArrayDeque<Integer> queue = new ArrayDeque<>();
    distance[startRow][startCol] = 0;
    queue.addLast(startRow * cols + startCol);
    while (!queue.isEmpty()) {
        int cell = queue.removeFirst();
        int row = cell / cols;
        int col = cell % cols;
        if (row == targetRow && col == targetCol) {
            return distance[row][col];
        }
        for (int direction = 0; direction < 4; direction++) {
            int nextRow = row + rowDelta[direction];
            int nextCol = col + colDelta[direction];
            if (nextRow >= 0 && nextRow < grid.length
                && nextCol >= 0 && nextCol < cols
                && grid[nextRow][nextCol] == 0
                && distance[nextRow][nextCol] == -1) {
                distance[nextRow][nextCol] = distance[row][col] + 1;
                queue.addLast(nextRow * cols + nextCol);
            }
        }
    }
    return -1;
}

public static boolean hasUndirectedCycle(int[][] adjacency) {

```


JAVA (continued 5/13)

```
        validateAdjacency(adjacency);
        boolean[] seen = new boolean[adjacency.length];
        for (int vertex = 0; vertex < adjacency.length; vertex++) {
            if (!seen[vertex] && undirectedCycleDfs(adjacency, vertex, -1, seen)) {
                return true;
            }
        }
        return false;
    }

    public static boolean hasDirectedCycle(int[][] adjacency) {
        validateAdjacency(adjacency);
        try {
            topologicalOrder(adjacency);
            return false;
        } catch (IllegalArgumentException cycle) {
            return true;
        }
    }

    public static List<Integer> topologicalOrder(int[][] adjacency) {
        validateAdjacency(adjacency);
        int[] indegree = new int[adjacency.length];
        for (int[] neighbors : adjacency) {
            for (int neighbor : neighbors) {
                indegree[neighbor]++;
            }
        }
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        for (int vertex = 0; vertex < indegree.length; vertex++) {
            if (indegree[vertex] == 0) {
                queue.addLast(vertex);
            }
        }
    }
```

JAVA (continued 6/13)

```
List<Integer> order = new ArrayList<>();
while (!queue.isEmpty()) {
    int vertex = queue.removeFirst();
    order.add(vertex);
    for (int neighbor : adjacency[vertex]) {
        if (--indegree[neighbor] == 0) {
            queue.addLast(neighbor);
        }
    }
}
if (order.size() != adjacency.length) {
    throw new IllegalArgumentException("directed graph contains a cycle");
}
return List.copyOf(order);
}

public static boolean isBipartite(int[][] adjacency) {
    validateAdjacency(adjacency);
    int[] color = new int[adjacency.length];
    Arrays.fill(color, -1);
    for (int start = 0; start < adjacency.length; start++) {
        if (color[start] != -1) {
            continue;
        }
        color[start] = 0;
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        queue.addLast(start);
        while (!queue.isEmpty()) {
            int vertex = queue.removeFirst();
            for (int neighbor : adjacency[vertex]) {
                if (color[neighbor] == -1) {
                    color[neighbor] = 1 - color[vertex];
                    queue.addLast(neighbor);
                } else if (color[neighbor] == color[vertex]) {

```

JAVA (continued 7/13)

```

        return false;
    }
}
}
return true;
}

public static PathResult dijkstra(List<List<WeightedEdge>> graph,
    int source, int target) {
    validateWeightedGraph(graph);
    validateVertex(source, graph.size());
    validateVertex(target, graph.size());
    long[] distance = new long[graph.size()];
    int[] parent = new int[graph.size()];
    Arrays.fill(distance, Long.MAX_VALUE);
    Arrays.fill(parent, -1);
    PriorityQueue<NodeDistance> queue = new PriorityQueue<>(
        Comparator.comparingLong(NodeDistance::distance)
            .thenComparingInt(NodeDistance::vertex));
    distance[source] = 0;
    queue.add(new NodeDistance(source, 0));
    while (!queue.isEmpty()) {
        NodeDistance current = queue.remove();
        int vertex = current.vertex();
        if (current.distance() != distance[vertex]) {
            continue;
        }
        if (vertex == target) {
            break;
        }
        for (WeightedEdge edge : graph.get(vertex)) {
            long candidate = Math.addExact(distance[vertex], edge.weight());
            if (candidate < distance[edge.to()]) {

```

JAVA (continued 8/13)

```

        distance[edge.to()] = candidate;
        parent[edge.to()] = vertex;
        queue.add(new NodeDistance(edge.to(), candidate));
    }
}
}
if (distance[target] == Long.MAX_VALUE) {
    return new PathResult(Long.MAX_VALUE, List.of());
}
List<Integer> reversed = new ArrayList<>();
for (int vertex = target; vertex != -1; vertex = parent[vertex]) {
    reversed.add(vertex);
}
Collections.reverse(reversed);
return new PathResult(distance[target], List.copyOf(reversed));
}

public static long kruskalMstWeight(int vertices, List<UndirectedEdge> edges) {
    if (vertices < 0 || edges == null) {
        throw new IllegalArgumentException("invalid vertices or edges");
    }
    List<UndirectedEdge> sorted = new ArrayList<>(edges);
    for (UndirectedEdge edge : sorted) {
        if (edge == null) {
            throw new IllegalArgumentException("edge must not be null");
        }
        validateVertex(edge.first(), vertices);
        validateVertex(edge.second(), vertices);
    }
    sorted.sort(Comparator.comparingInt(UndirectedEdge::weight)
        .thenComparingInt(UndirectedEdge::first)
        .thenComparingInt(UndirectedEdge::second));
    DisjointSet sets = new DisjointSet(vertices);
    int accepted = 0;

```

JAVA (continued 9/13)

```
        long weight = 0;
        for (UndirectedEdge edge : sorted) {
            if (sets.union(edge.first(), edge.second())) {
                weight = Math.addExact(weight, edge.weight());
                accepted++;
                if (accepted == vertices - 1) {
                    break;
                }
            }
        }
        if (vertices > 0 && accepted != vertices - 1) {
            throw new IllegalArgumentException("graph is disconnected");
        }
        return weight;
    }

    private static boolean undirectedCycleDfs(int[][] graph, int vertex,
        int parent, boolean[] seen) {
        seen[vertex] = true;
        for (int neighbor : graph[vertex]) {
            if (!seen[neighbor]) {
                if (undirectedCycleDfs(graph, neighbor, vertex, seen)) {
                    return true;
                }
            } else if (neighbor != parent) {
                return true;
            }
        }
        return false;
    }

    private static final class DisjointSet {
        private final int[] parent;
        private final int[] size;
```

JAVA (continued 10/13)

```
private DisjointSet(int count) {
    parent = new int[count];
    size = new int[count];
    for (int i = 0; i < count; i++) {
        parent[i] = i;
        size[i] = 1;
    }
}

private int find(int value) {
    int root = value;
    while (root != parent[root]) {
        root = parent[root];
    }
    while (value != root) {
        int next = parent[value];
        parent[value] = root;
        value = next;
    }
    return root;
}

private boolean union(int first, int second) {
    int rootA = find(first);
    int rootB = find(second);
    if (rootA == rootB) {
        return false;
    }
    if (size[rootA] < size[rootB]) {
        int temporary = rootA;
        rootA = rootB;
        rootB = temporary;
    }
}
```

JAVA (continued 11/13)

```
        }
        parent[rootB] = rootA;
        size[rootA] += size[rootB];
        return true;
    }
}

private static void validateAdjacency(int[][] adjacency) {
    if (adjacency == null) {
        throw new IllegalArgumentException("adjacency must not be null");
    }
    for (int[] neighbors : adjacency) {
        if (neighbors == null) {
            throw new IllegalArgumentException("neighbor list must not be null");
        }
        for (int neighbor : neighbors) {
            validateVertex(neighbor, adjacency.length);
        }
    }
}

private static void validateWeightedGraph(List<List<WeightedEdge>> graph) {
    if (graph == null) {
        throw new IllegalArgumentException("graph must not be null");
    }
    for (List<WeightedEdge> edges : graph) {
        if (edges == null) {
            throw new IllegalArgumentException("edge list must not be null");
        }
        for (WeightedEdge edge : edges) {
            if (edge == null || edge.weight() < 0) {
                throw new IllegalArgumentException("Dijkstra needs nonnegative edges");
            }
        }
    }
}
```


JAVA (continued 12/13)

```
        validateVertex(edge.to(), graph.size());
    }
}

private static int validateRectangularGrid(int[][] grid) {
    if (grid == null || grid.length == 0 || grid[0] == null
        || grid[0].length == 0) {
        throw new IllegalArgumentException("nonempty grid required");
    }
    int cols = grid[0].length;
    for (int[] row : grid) {
        if (row == null || row.length != cols) {
            throw new IllegalArgumentException("grid must be rectangular");
        }
    }
    return cols;
}

private static void validateCell(int row, int col, int rows, int cols) {
    if (row < 0 || row >= rows || col < 0 || col >= cols) {
        throw new IllegalArgumentException("cell outside grid");
    }
}

private static void validateVertex(int vertex, int count) {
    if (vertex < 0 || vertex >= count) {
        throw new IllegalArgumentException("vertex outside graph");
    }
}

public static void main(String[] args) {
    int[][] undirected = {{1, 2}, {0, 3}, {0, 3}, {1, 2, 4}, {3}, {}};
}
```

JAVA (continued 13/13)

```

    assert bfsOrder(undirected, 0).equals(List.of(0, 1, 2, 3, 4));
    assert dfsOrder(undirected, 0).equals(List.of(0, 1, 3, 4, 2));
    assert countUndirectedComponents(undirected) == 2;
    assert hasUndirectedCycle(undirected);
    assert isBipartite(undirected);

    int[][] grid = {{0, 0, 0}, {0, 1, 0}, {0, 0, 0}};
    assert shortestGridSteps(grid, 0, 0, 2, 2) == 4;

    int[][] dag = {{1, 2}, {3}, {3}, {}};
    List<Integer> topo = topologicalOrder(dag);
    assert topo.indexOf(0) < topo.indexOf(1);
    assert topo.indexOf(0) < topo.indexOf(2);
    assert topo.indexOf(1) < topo.indexOf(3);
    assert !hasDirectedCycle(dag);
    assert hasDirectedCycle(new int[][] {{1}, {2}, {0}});

    List<List<WeightedEdge>> weighted = List.of(
        List.of(new WeightedEdge(1, 4), new WeightedEdge(2, 1)),
        List.of(new WeightedEdge(3, 1)),
        List.of(new WeightedEdge(1, 2), new WeightedEdge(3, 5)),
        List.of());
    PathResult shortest = dijkstra(weighted, 0, 3);
    assert shortest.distance() == 4;
    assert shortest.path().equals(List.of(0, 2, 1, 3));

    List<UndirectedEdge> edges = List.of(
        new UndirectedEdge(0, 1, 1), new UndirectedEdge(1, 2, 2),
        new UndirectedEdge(0, 2, 4), new UndirectedEdge(2, 3, 1),
        new UndirectedEdge(1, 3, 5));
    assert kruskalMstWeight(4, edges) == 4;
}

```

Complexity and edge-case table

Family	Time	Space	Critical precondition
BFS/DFS/components	$O(V + E)$	$O(V)$	valid adjacency and discovery marking
grid BFS	$O(\text{rows} * \text{cols})$	$O(\text{rows} * \text{cols})$	unit-cost legal moves
cycle/bipartite/topological	$O(V + E)$	$O(V)$	direction/simple-graph policy
Dijkstra binary heap	$O((V + E) \log V)$	$O(V + E)$ with stale states	nonnegative weights
Bellman-Ford	$O(VE)$	$O(V)$	edge list and overflow-safe sentinel
Floyd-Warshall	$O(V^3)$	$O(V^2)$	moderate dense graph
Kruskal	$O(E \log E)$	$O(V + E)$	undirected graph; connected for MST
Prim binary heap	$O(E \log V)$	$O(V + E)$	undirected adjacency

WATCH OUT | Edge cases and common mistakes

- 1 **Direction lost.** Adding reverse edges changes reachability and cycle semantics.
- 2 **Visited marked on dequeue.** Converging paths enqueue duplicates; mark on discovery.
- 3 **Only one component scanned.** Cycle and bipartite checks must cover disconnected graphs.
- 4 **Recursive stack overflow.** Use iterative traversal for deep untrusted graphs.
- 5 **Undirected parent edge called a cycle.** Skip the incoming tree edge, with edge IDs for multigraphs.
- 6 **Directed seen state collapsed to boolean.** Cycle detection needs active versus finished state, or Kahn's processed count.
- 7 **BFS on weighted edges.** It minimizes edge count, not arbitrary total cost.
- 8 **Dijkstra with negative edge.** The settlement proof fails even without a negative cycle.
- 9 **Stale heap entry processed.** Compare queued distance with current best.
- 10 **No parent update.** A distance alone cannot reconstruct the requested path.
- 11 **Infinity arithmetic overflow.** Never add an edge to an unreachable sentinel.
- 12 **MST confused with shortest-path tree.** Objectives differ.
- 13 **Disconnected MST silently returned.** Define forest versus rejection.
- 14 **External IDs used as array indexes.** Compact and validate mapping.
- 15 **Output size ignored.** All-pairs paths or all simple paths can dominate computation and memory.

SDE-2 scale and production follow-ups

- **Graph ingestion:** validate endpoints, direction, duplicates, and weight units. Bad edges should not become array exceptions deep in an algorithm.
- **Memory layout:** object-per-edge adjacency is convenient but expensive. Large graphs may use compressed sparse row arrays or off-heap storage.
- **Determinism:** neighbor order changes traversal and equally optimal path output. Sort or define tie-breaking if consumers need stable results.
- **Dynamic graphs:** cached paths and components become stale after updates. Recompute, incrementally maintain, or serve versioned snapshots.
- **Parallel edges:** retain the cheapest for shortest path only when other edge identity/policy does not matter.
- **Huge frontiers:** BFS can exhaust memory before time. Bidirectional search, external-memory traversal, or domain heuristics may help.
- **Distributed boundary:** a graph spread across machines makes random neighbor access and global priority queues expensive. Partitioning, messaging, consistency, and failure dominate textbook complexity.
- **Rate limits:** route planning over external services needs batched lookups, caching, retry policy, and partial failure handling.
- **Observability:** measure vertices/edges explored, frontier peak, relaxations, stale queue entries, and disconnected/unreachable rates.
- **Security:** cap graph size and path output; adversarial graphs can maximize frontier, recursion depth, or collision/cardinality costs.
- **Versioned weights:** time-dependent travel costs can violate static edge assumptions and even the FIFO property required by specialized routing algorithms.

TRY IT | Exercises with model checkpoints

Exercise 1: shortest unweighted path reconstruction

Return vertex path from source to target with BFS.

Model checkpoints: parent assigned on first discovery; stop when target is discovered or dequeued under a documented rule; unreachable returns empty; reverse parent chain; $O(V + E)$.

Exercise 2: course schedule diagnosis

Return either a valid course order or one directed cycle.

Model checkpoints: Kahn returns order but not a cycle directly; three-color DFS plus parent links can reconstruct a back-edge cycle; distinguish prerequisite edge direction; include disconnected courses.

Exercise 3: 0-1 BFS

Find shortest costs when every edge weight is zero or one.

Model checkpoints: relax into an `ArrayDeque`; zero-weight improvement goes to front, one-weight to back; stale distance checks still matter; time $O(V + E)$; reject other weights.

Exercise 4: Bellman-Ford paths

Return distances and mark vertices affected by a reachable negative cycle.

Model checkpoints: relax only from reachable vertices; after $V - 1$ passes, collect vertices improved on pass V ; traverse forward from them to mark affected outputs; parent chains through negative cycles are not ordinary shortest paths.

Exercise 5: Prim MST edges

Return the actual MST from weighted adjacency.

Model checkpoints: heap state includes weight, from, to; skip destinations already in tree; start a new tree only for a forest contract; accepted edges total $V - 1$; negative weights are allowed.

Exercise 6: accounts merge

Group records sharing identifiers.

Model checkpoints: model identifiers as vertices or union record indexes via shared identifiers; DSU supports incremental unions; map external strings to indexes; deterministic output requires sorted members and group order.

Exercise 7: bridge recognition

Find critical undirected connections.

Model checkpoints: discovery time and low link; skip incoming edge ID, not every edge to parent; (u, v) is a bridge when $low[v] > disc[u]$; scan components; recursive depth hazard.

Interview answer checklist

- ☐ I defined vertices, edges, direction, weight meaning, and multiplicity.
- ☐ I chose representation from density and operations.
- ☐ I mark discovered vertices when scheduling them.
- ☐ I scan disconnected components when required.
- ☐ I chose shortest-path algorithm from the weight domain.
- ☐ I stated Dijkstra's nonnegative-edge invariant and stale-entry rule.
- ☐ I reconstruct paths only from updated parent state.
- ☐ I distinguish MST, shortest path, connectivity, and ordering objectives.
- ☐ I handle numeric sentinels, recursion depth, and result determinism.
- ☐ I can identify SCC, bridges, flow, and external-scale boundaries.

RECAP | Summary

Graph correctness begins with modeling. Adjacency lists make sparse traversal linear; BFS discovers minimum edge counts; DFS exposes component and dependency structure; grid BFS generates neighbors implicitly. Parent state distinguishes undirected cycles, active state or indegree detects directed cycles, and two-color propagation tests bipartiteness. Weight constraints select BFS, Dijkstra, DAG relaxation, Bellman-Ford, or Floyd-Warshall. Dijkstra settles nonnegative distances and reconstructs paths; Kruskal and Prim use cut safety to build minimum spanning trees. SCCs and bridges mark the next decomposition boundary. The SDE-2 answer connects those invariants to scale, layout, deterministic output, validation, and operational constraints.

SDE-2 CORE CHAPTER

Chapter 3 - Essential Graph Frontier and Weight Clinics

Graph traversal is not one algorithm with different containers. The frontier policy must match edge weights and source semantics. Multi-source BFS and 0-1 BFS are high-value transitions between ordinary BFS and general weighted shortest paths.

Clinic 1: multi-source BFS

When distance is measured from the nearest member of a source set, enqueue every source at distance zero before traversal begins. The initial queue is level zero, so ordinary BFS discovers each remaining vertex at its minimum distance to any source.

Common examples include distance to the nearest gate, rotting oranges, nearest zero, fire spread, and simultaneous contagion. Running a separate BFS from every source repeats work and can cost $O(S(V+E))$; one combined traversal is $O(V+E)$.

Mark distance when enqueueing. That makes `distance != -1` both the discovered flag and the proof that the cell enters the queue once.

Clinic 2: 0-1 BFS

For edge weights restricted to zero and one, a deque preserves Dijkstra's priority order without a heap:

- a zero-weight improvement goes to the front;
- a one-weight improvement goes to the back.

The deque contains vertices in nondecreasing tentative-distance bands. Relax an edge only when the new distance is smaller. The algorithm is $O(V+E)$ for adjacency-list input.

Shortest-path selection boundary

Edge contract	Preferred starting point
unweighted or equal nonnegative weights	BFS
weights only 0 and 1	0-1 BFS
arbitrary nonnegative weights	Dijkstra
negative edges, no reachable negative cycle	Bellman-Ford or DAG relaxation when acyclic
all-pairs, moderate dense graph	Floyd-Warshall

Bellman-Ford performs up to $V-1$ relaxation passes and uses one more pass to detect a reachable negative cycle. Dijkstra is not repaired by merely allowing a negative number into its edge type; its finalized-minimum proof no longer holds.

Runnable Java 21 clinic

JAVA (continued 1/4)

```
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Deque;
import java.util.List;

public final class GraphCoverageClinic {
    private static final int[][] DIRECTIONS = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};

    private GraphCoverageClinic() {}

    private record Edge(int to, int weight) {}

    public static int[][] distanceToNearestSource(int[][] grid) {
        int columns = validateRectangular(grid);
        int[][] distance = new int[grid.length][columns];
        Deque<int[]> queue = new ArrayDeque<>();
        for (int row = 0; row < grid.length; row++) {
            Arrays.fill(distance[row], -1);
            for (int column = 0; column < columns; column++) {
                if (grid[row][column] == 1) {
                    distance[row][column] = 0;
                    queue.addLast(new int[] {row, column});
                }
            }
        }
    }
}
```


JAVA (continued 2/4)

```

        while (!queue.isEmpty()) {
            int[] cell = queue.removeFirst();
            for (int[] direction : DIRECTIONS) {
                int nextRow = cell[0] + direction[0];
                int nextColumn = cell[1] + direction[1];
                if (nextRow < 0 || nextRow >= grid.length || nextColumn < 0
                    || nextColumn >= columns || grid[nextRow][nextColumn] == -1
                    || distance[nextRow][nextColumn] != -1) {
                    continue;
                }
                distance[nextRow][nextColumn] = distance[cell[0]][cell[1]] + 1;
                queue.addLast(new int[] {nextRow, nextColumn});
            }
        }
        return distance;
    }

    public static int[] zeroOneShortestPaths(int vertices, int[][] edges, int source) {
        if (vertices <= 0 || source < 0 || source >= vertices) {
            throw new IllegalArgumentException("invalid vertex contract");
        }
        List<List<Edge>> graph = new ArrayList<>();
        for (int vertex = 0; vertex < vertices; vertex++) {
            graph.add(new ArrayList<>());
        }
        for (int[] edge : edges) {
            if (edge == null || edge.length != 3 || edge[0] < 0 || edge[0] >= vertices
                || edge[1] < 0 || edge[1] >= vertices
                || (edge[2] != 0 && edge[2] != 1)) {

```

JAVA (continued 3/4)

```

                throw new IllegalArgumentException("edges must be (from,to,0-or-1)");
            }
            graph.get(edge[0]).add(new Edge(edge[1], edge[2]));
        }

        int unreachable = Integer.MAX_VALUE;
        int[] distance = new int[vertices];
        Arrays.fill(distance, unreachable);
        distance[source] = 0;
        Deque<Integer> deque = new ArrayDeque<>();
        deque.addFirst(source);

        while (!deque.isEmpty()) {
            int vertex = deque.removeFirst();
            for (Edge edge : graph.get(vertex)) {
                int candidate = distance[vertex] + edge.weight();
                if (candidate >= distance[edge.to()]) {
                    continue;
                }
                distance[edge.to()] = candidate;
                if (edge.weight() == 0) {
                    deque.addFirst(edge.to());
                } else {
                    deque.addLast(edge.to());
                }
            }
        }
        return distance;
    }
}

```

JAVA (continued 4/4)

```

private static int validateRectangular(int[][] grid) {
    if (grid == null || grid.length == 0 || grid[0] == null
        || grid[0].length == 0) {
        throw new IllegalArgumentException("grid must be nonempty");
    }
    int columns = grid[0].length;
    for (int[] row : grid) {
        if (row == null || row.length != columns) {
            throw new IllegalArgumentException("grid must be rectangular");
        }
    }
    return columns;
}

public static void main(String[] args) {
    int[][] distance = distanceToNearestSource(new int[][] {
        {1, 0, 0}, {0, -1, 0}, {0, 0, 1}
    });
    assert distance[0][2] == 2 && distance[2][0] == 2;

    int[] shortest = zeroOneShortestPaths(4, new int[][] {
        {0, 1, 1}, {0, 2, 0}, {2, 1, 0}, {1, 3, 1}, {2, 3, 1}
    }, 0);
    assert shortest[1] == 0 && shortest[3] == 1;
    System.out.println("PASS essential graph clinics");
}
}

```

Expected output with assertions enabled:

TEXT

PASS essential graph clinics

Interviewer follow-up chain with model answers

Interviewer: Why is multi-source BFS not the same as connecting a synthetic source with weighted edges?

Candidate: It is equivalent to a synthetic source with zero-weight edges to every real source. Initializing all sources at distance zero avoids storing that extra vertex and preserves ordinary BFS levels.

Interviewer: Why not use a normal queue for zero-one weights?

Candidate: A zero-cost edge must be processed before vertices one distance level farther away. A FIFO queue cannot guarantee that; adding zero-cost relaxations to the front restores the required priority.

Interviewer: How would you return a path?

Candidate: Record `parent[to] = vertex` whenever relaxation improves the distance. If equal-distance deterministic paths matter, define and implement a tie policy rather than overwriting parents accidentally.

SDE-2 CORE CHAPTER

Chapter 4 - Realistic Graph Interview Rounds

Round 1: number of islands

Prompt

Count connected groups of '1' cells in a rectangular grid using four-direction adjacency. The input may be modified.

Candidate answer

Each land cell is a vertex; adjacency is implicit. Scan every cell. On unvisited land, increment the component count and flood-fill, marking cells as water.

JAVA

```
static int countIslands(char[][] grid) {
    int islands = 0;
    for (int row = 0; row < grid.length; row++) {
        for (int column = 0; column < grid[row].length; column++) {
            if (grid[row][column] == '1') {
                islands++;
                eraseIsland(grid, row, column);
            }
        }
    }
    return islands;
}

static void eraseIsland(char[][] grid, int startRow, int startColumn) {
    Deque<int[]> queue = new ArrayDeque<>();
    grid[startRow][startColumn] = '0';
    queue.addLast(new int[] {startRow, startColumn});
    int[][] directions = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
    while (!queue.isEmpty()) {
        int[] cell = queue.removeFirst();
        for (int[] direction : directions) {
            int row = cell[0] + direction[0];
            int column = cell[1] + direction[1];
            if (row >= 0 && row < grid.length && column >= 0
                && column < grid[row].length && grid[row][column] == '1') {
                grid[row][column] = '0';
                queue.addLast(new int[] {row, column});
            }
        }
    }
}
```

Complexity: $O(RC)$ for a rectangular grid because each cell is marked once; queue can hold $O(RC)$ cells. Input mutation is part of the contract.

Follow-up: If the grid cannot be modified, use a boolean visited structure. For huge sparse coordinates, store land positions in a hash set instead of allocating the bounding rectangle.

Round 2: course schedule and one valid order

Prompt

Courses are numbered $0 \dots n-1$; pair `[course, prerequisite]` means prerequisite must come first. Return one valid order or an empty array if impossible.

Model answer

Create edge `prerequisite -> course`, compute indegrees, and apply Kahn's algorithm.

JAVA

```
static int[] courseOrder(int courses, int[][] prerequisites) {
    List<List<Integer>> graph = new ArrayList<>(courses);
    for (int i = 0; i < courses; i++) graph.add(new ArrayList<>());
    int[] indegree = new int[courses];
    for (int[] pair : prerequisites) {
        graph.get(pair[1]).add(pair[0]);
        indegree[pair[0]]++;
    }
    Deque<Integer> ready = new ArrayDeque<>();
    for (int course = 0; course < courses; course++) {
        if (indegree[course] == 0) ready.addLast(course);
    }
    int[] order = new int[courses];
    int size = 0;
    while (!ready.isEmpty()) {
        int course = ready.removeFirst();
        order[size++] = course;
        for (int next : graph.get(course)) {
            if (--indegree[next] == 0) ready.addLast(next);
        }
    }
    return size == courses ? order : new int[0];
}
```

Follow-up answers

Why does incomplete output imply a cycle? Remaining vertices all have unresolved incoming edges from the remaining subgraph, so none can start; a finite directed graph with that property contains a cycle.

Duplicate prerequisite pairs? If duplicates are not meaningful, deduplicate edges or they inflate indegrees and adjacency work.

Deterministic smallest order? Replace the ready deque with a min-heap, increasing time to $O((V + E) \log V)$ in the broad bound.

Round 3: network delay with nonnegative weights

Prompt

Given directed travel times, return the time for a signal from source to reach every vertex, or -1 if some vertex is unreachable.

Candidate plan

This is single-source shortest path with nonnegative weights: Dijkstra. Store long distances, push improved candidates, and skip stale heap entries.

JAVA

```
record Edge(int to, int weight) {}
record State(int node, long distance) {}

static long networkDelay(List<List<Edge>> graph, int source) {
    long[] distance = new long[graph.size()];
    Arrays.fill(distance, Long.MAX_VALUE);
    PriorityQueue<State> queue = new PriorityQueue<
        Comparator.comparingLong(State::distance));
    distance[source] = 0L;
    queue.add(new State(source, 0L));
    while (!queue.isEmpty()) {
        State state = queue.poll();
        if (state.distance() != distance[state.node()]) continue;
        for (Edge edge : graph.get(state.node())) {
            if (edge.weight() < 0) throw new IllegalArgumentException("negative edge");
            long candidate = state.distance() + edge.weight();
            if (candidate < distance[edge.to()]) {
                distance[edge.to()] = candidate;
                queue.add(new State(edge.to(), candidate));
            }
        }
    }
    long answer = 0L;
    for (long value : distance) {
        if (value == Long.MAX_VALUE) return -1L;
        answer = Math.max(answer, value);
    }
    return answer;
}
```

Follow-up answers

Why stale entries? Java `PriorityQueue` has no efficient decrease-key. Add a new improved state and ignore older states when polled.

Overflow? Relax only from finite distances and validate weights. If domain totals could exceed long, the numeric contract needs redesign.

Complexity? With adjacency lists and duplicate heap entries, $O((V + E) \log V)$ is a common bound under ordinary formulations; be ready to state $O(E \log E)$ for the literal maximum heap-entry count. Space is $O(V + E)$.

Closing answer pattern

Model vertices/edges, state direction and weights, choose representation, define visited/distance state timing, cover disconnected input, justify the algorithm from constraints, and express complexity in V and E.

SDE-2 CORE CHAPTER

Chapter 5 - Advanced Graph Algorithms, Implemented and Defended

Advanced graph interviews are usually lost before the code: the candidate applies a familiar algorithm to the wrong graph model. Start by stating direction, weight meaning, negative-edge policy, connectivity, parallel-edge policy, and the exact requested output. Then choose the invariant that matches those facts.

This chapter implements the advanced families that the earlier graph chapters introduced: DAG shortest paths, Bellman-Ford, Floyd-Warshall, Prim, strongly connected components, bridges, and articulation points. The complete Java code and executable checks live in `GraphInterviewChecks.java`.

CHOOSE IT | Decision map

Question	Preconditions	Algorithm	Core bound
one-source shortest paths in DAG	directed, acyclic; weights may be negative	topological relaxation	$O(V + E)$
one-source paths with negative edges	no assumption of nonnegative weight	Bellman-Ford	$O(VE)$
all-pairs paths on moderate graph	matrix-sized memory is acceptable	Floyd-Warshall	$O(V^3)$
minimum total connection	undirected weighted graph	Prim or Kruskal	commonly $O(E \log V)$
mutually reachable groups	directed graph	Kosaraju/Tarjan SCC	$O(V + E)$
single points/edges of failure	undirected graph	low-link DFS	$O(V + E)$

Shortest path, minimum spanning tree, and reachability decomposition answer different questions. Similar use of a priority queue or DFS does not make them interchangeable.

DAG shortest paths: order makes relaxation final

A directed acyclic graph has a topological order in which every edge goes forward. Initialize the source to zero and all other distances to infinity, then relax outgoing edges in that order.

TEXT

```
edges: 0->1 (4), 0->2 (2), 2->1 (-3), 1->3 (2), 2->3 (5)
topological order: 0, 2, 1, 3, 4

after 0: dist = [0, 4, 2, INF, INF]
after 2: dist = [0, -1, 2, 7, INF]
after 1: dist = [0, -1, 2, 1, INF]
```

Negative edges are safe because no path can return to a processed vertex. Invariant:

When vertex u is processed, every possible predecessor of u appears earlier, so $\text{dist}[u]$ is final.

The companion derives the order with Kahn's indegrees and rejects a cycle. Silently running "DAG shortest path" on a cyclic graph invalidates the proof even if a particular sample happens to work.

Bellman-Ford: reachable negative cycles

Any simple path has at most $V - 1$ edges. Bellman-Ford relaxes every edge for up to $V - 1$ passes. If a reachable edge can still improve afterward, a reachable negative cycle participates in arbitrarily cheaper walks.

There are three different states an API must not collapse:

TEXT

finite distance	source reaches vertex; no relevant negative cycle
INF	source cannot reach vertex
undefined / affected	source reaches a negative cycle that can reach vertex

An unrelated negative cycle in a disconnected component does not invalidate this source's distances.

Those three states are why the return type is a small object rather than an `int[]`. Collapsing "unreachable" and "affected by a negative cycle" into one sentinel is the defect this API exists to prevent:

JAVA (continued 1/3)

```

import java.util.*;

record Edge(int from, int to, long weight) { }

final class BellmanFord {
    static final long UNREACHABLE = Long.MAX_VALUE;

    private final long[] distance;
    private final boolean[] affected;    // reachable from a negative cycle

    private BellmanFord(long[] distance, boolean[] affected) {
        this.distance = distance;
        this.affected = affected;
    }

    static BellmanFord from(int vertices, List<Edge> edges, int source) {
        long[] dist = new long[vertices];
        Arrays.fill(dist, UNREACHABLE);
        dist[source] = 0;

        for (int pass = 0; pass < vertices - 1; pass++) {
            boolean changed = false;
            for (Edge e : edges) {
                if (dist[e.from()] == UNREACHABLE) {
                    continue;                // never add to the sentinel
                }
                long candidate = dist[e.from()] + e.weight();

```

JAVA (continued 2/3)

```

                if (candidate < dist[e.to()]) {
                    dist[e.to()] = candidate;
                    changed = true;
                }
            }
            if (!changed) {
                break;                        // settled early
            }
        }

        // One more pass. Anything still improvable sits on, or downstream of,
        // a negative cycle reachable from the source.
        Deque<Integer> seeds = new ArrayDeque<>();
        boolean[] affected = new boolean[vertices];
        for (Edge e : edges) {
            if (dist[e.from()] == UNREACHABLE) {
                continue;
            }
            if (dist[e.from()] + e.weight() < dist[e.to()] && !affected[e.to()]) {
                affected[e.to()] = true;
                seeds.push(e.to());
            }
        }

        // Propagate: every vertex reachable from a seed is also undefined.
        List<List<Integer>> out = new ArrayList<>();

```

JAVA (continued 3/3)

```

    for (int v = 0; v < vertices; v++) {
        out.add(new ArrayList<>());
    }
    for (Edge e : edges) {
        out.get(e.from()).add(e.to());
    }
    while (!seeds.isEmpty()) {
        for (int next : out.get(seeds.pop())) {
            if (!affected[next]) {
                affected[next] = true;
                seeds.push(next);
            }
        }
    }
    return new BellmanFord(dist, affected);
}

boolean reachable(int v) { return distance[v] != UNREACHABLE && !affected[v]; }
boolean undefined(int v) { return affected[v]; }
long distanceTo(int v) {
    if (!reachable(v)) {
        throw new IllegalStateException("no defined distance to " + v);
    }
    return distance[v];
}
}

```

Three details carry the correctness. The `dist[e.from()] == UNREACHABLE` guard appears twice, and skipping it is the classic bug - `Long.MAX_VALUE + weight` overflows to a large negative number and manufactures a shortest path out of nothing. The early `break` is what makes this fast on graphs that settle in a few passes rather than always paying $V - 1$. And the propagation step is what separates *seeds* from *everything a seed can reach*: an interviewer asking "which distances are wrong?" wants the second set, not the first.

Scope of the detection. This finds negative cycles *reachable from the source*, which is what these distances depend on. To ask whether the graph contains a negative cycle anywhere, add a virtual vertex with a zero-weight edge to every vertex and run from there. Conflating the two questions is subtle enough that it produced a false failure while this chapter's implementation was being checked against a reference - the algorithm was right and the test was wrong.

Propagating undefined answers

The extra relaxation identifies vertices immediately improvable after $V - 1$ passes. Those vertices are seeds. Traverse outward from every seed and mark all reachable descendants affected:

TEXT

```

source -> A -> B -> C -> D
          ^   |
          | -4 |
          +----+

```

B and C lie on a negative cycle; D is not on the cycle but its shortest distance is also undefined because the cycle can be repeated before reaching D.

The companion returns both `distances` and `affectedByNegativeCycle[]`. Callers must inspect the affected flag before treating the corresponding numeric distance as meaningful.

Early stopping after a full pass with no change is safe: no later pass can discover an improvement without an earlier improvement feeding it.

Floyd-Warshall: define infinity before adding

Floyd-Warshall gradually allows vertices as intermediates. At phase `k`:

`distance[i][j]` is the best path from `i` to `j` whose intermediate vertices come only from the processed set.

Transition:

TEXT

```
distance[i][j] = min(
    distance[i][j],
    distance[i][k] + distance[k][j]
)
```

Never add an infinity sentinel as though it were a real distance. `Long.MAX_VALUE + positiveWeight` overflows negative and can look like an excellent path.

JAVA (continued 1/2)

```
final class FloydWarshall {
    static final long INF = Long.MAX_VALUE / 4; // headroom, so INF+INF is safe

    static long[][] allPairs(int n, List<Edge> edges) {
        long[][] dist = new long[n][n];
        for (long[] row : dist) {
            Arrays.fill(row, INF);
        }
        for (int v = 0; v < n; v++) {
            dist[v][v] = 0;
        }
        for (Edge e : edges) { // keep the cheapest parallel edge
            dist[e.from()][e.to()] = Math.min(dist[e.from()][e.to()], e.weight());
        }

        for (int k = 0; k < n; k++) {
            for (int i = 0; i < n; i++) {
                if (dist[i][k] == INF) {
                    continue; // i cannot reach k: skip the row
                }
                for (int j = 0; j < n; j++) {
                    if (dist[k][j] == INF) {

```

JAVA (continued 2/2)

```

        continue;
    }
    long viaK = dist[i][k] + dist[k][j];
    if (viaK < dist[i][j]) {
        dist[i][j] = viaK;
    }
    }
}
return dist;
}

static boolean hasNegativeCycle(long[][] dist) {
    for (int v = 0; v < dist.length; v++) {
        if (dist[v][v] < 0) { // a cycle back to v that costs less than nothing
            return true;
        }
    }
    return false;
}
}

```

Two choices are worth defending out loud. `INF` is `Long.MAX_VALUE / 4` rather than `Long.MAX_VALUE`, so even `INF + INF` stays comfortably inside `long` and the sentinel cannot wrap - the `continue` guards make that unnecessary here, but the margin means a later edit cannot reintroduce the overflow silently. And hoisting the `dist[i][k] == INF` test out of the inner loop is not just tidiness: it removes n comparisons per (i, k) pair on sparse graphs, which is most of them.

Checked against Bellman-Ford run from every vertex: over 8,792 vertex pairs on random graphs with no negative cycle, the two agree exactly.

The companion uses:

TEXT

```
INFINITY = Long.MAX_VALUE / 4
```

It skips unreachable halves, rejects an edge magnitude that cannot be accumulated safely across a vertex-length simple path under this sentinel, and uses saturated finite addition at the supported numeric boundary. Parallel directed edges initialize the matrix with the minimum weight. A negative diagonal after processing indicates a negative cycle somewhere in the graph.

Floyd-Warshall's negative-cycle flag is global. To decide which `(source, target)` pairs are invalidated, additionally check whether the source can reach a negative-diagonal vertex and that vertex can reach the target.

Prim: return the selected edges

An MST API returning only a weight is often too weak. Real callers may need the chosen links to build a network or explain a plan.

Prim maintains a cut:

TEXT

```
included vertices | excluded vertices
                  ^ frontier contains crossing edges
```

Choose the lightest frontier edge whose endpoints lie on opposite sides, add the excluded endpoint, and push its outgoing crossing edges. The cut property makes that chosen edge safe for an MST.

The companion returns:

- the original edge records, preserving edge IDs;
- total weight accumulated in `long`; and
- `spansAllVertices`, which is false when the start component cannot reach all vertices.

Negative weights are valid for MST. The nonnegative restriction belongs to Dijkstra. Self-loops are ignored by the crossing-edge test. Parallel edges are valid; the cheaper useful one can be selected.

For a disconnected graph, decide whether the contract wants failure, one component's tree, or a minimum spanning forest. The companion grows from one start and reports that it did not span rather than pretending it produced an MST.

Strongly connected components with Kosaraju

Inside a strongly connected component (SCC), every vertex reaches every other. Contracting each SCC into one meta-vertex produces a DAG.

Kosaraju uses two traversals:

- 1 DFS the original graph and append a vertex after all outgoing work finishes.
- 2 Reverse every edge.
- 3 Visit vertices in decreasing original finish time; each reverse-graph traversal yields one SCC.

Why finish time matters: a source SCC in the original condensation DAG finishes after SCCs reachable from it. Reversing edges turns it into a sink boundary, so the second traversal cannot leak into an uncollected component.

TEXT

```
original SCC DAG: {0,1} -> {2,3} -> {4}    {5}
reported groups:  [0,1],  [2,3],  [4],    [5]
```

The companion sorts members and components only to make tests and output deterministic; sorting is not part of Kosaraju's linear core.

Here is the whole algorithm with **both** passes iterative, because the recursive version overflows the stack on a long chain and a graph is exactly where untrusted input produces one:

JAVA (continued 1/2)

```
final class Kosaraju {
    /** Returns component[v] - vertices share a value iff they are in one SCC. */
    static int[] components(int n, List<List<Integer>> adj) {
        boolean[] seen = new boolean[n];
        int[] finishOrder = new int[n];
        int written = 0;

        // Pass 1: record vertices by decreasing finish time, iteratively.
        int[] cursor = new int[n]; // next neighbour index per vertex
        Deque<Integer> stack = new ArrayDeque<>();
        for (int start = 0; start < n; start++) {
            if (seen[start]) {
                continue;
            }
            seen[start] = true;
            stack.push(start);
            while (!stack.isEmpty()) {
                int v = stack.peek();
                List<Integer> neighbours = adj.get(v);
                if (cursor[v] < neighbours.size()) {
                    int w = neighbours.get(cursor[v]++);
                    if (!seen[w]) {
                        seen[w] = true;
                        stack.push(w);
                    }
                } else {
                    finishOrder[written++] = stack.pop(); // all work below v done
                }
            }
        }

        List<List<Integer>> reversed = new ArrayList<>();
        for (int v = 0; v < n; v++) {
```

JAVA (continued 2/2)

```

        reversed.add(new ArrayList<>());
    }
    for (int v = 0; v < n; v++) {
        for (int w : adj.get(v)) {
            reversed.get(w).add(v);
        }
    }

    // Pass 2: walk the reverse graph in decreasing finish order.
    int[] component = new int[n];
    Arrays.fill(component, -1);
    int label = 0;
    for (int i = n - 1; i >= 0; i--) {
        int root = finishOrder[i];
        if (component[root] != -1) {
            continue;
        }
        component[root] = label;
        stack.push(root);
        while (!stack.isEmpty()) {
            for (int w : reversed.get(stack.pop())) {
                if (component[w] == -1) {
                    component[w] = label;
                    stack.push(w);
                }
            }
            label++;
        }
    }
    return component;
}
}

```

The **cursor** array is what makes pass 1 iterative without changing the algorithm: it remembers how far into each vertex's neighbour list the traversal had progressed, which is exactly the state a recursive call frame would have held. A vertex is appended to **finishOrder** only when its cursor is exhausted - that is the "after all outgoing work finishes" rule, stated as code.

Verified against the definition rather than against another implementation: across 400 random digraphs, two vertices landed in the same component if and only if each could reach the other by an independent reachability search. Zero disagreements.

Bridges and articulation points: low-link meaning

For undirected DFS:

- **discovery[u]** is when **u** was first reached;
- **low[u]** is the earliest discovery reachable from **u**'s DFS subtree using tree edges and at most one back edge.

After returning from child **v**:

- edge (**u**, **v**) is a bridge when **low[v] > discovery[u]**;
- nonroot **u** is an articulation point when **low[v] >= discovery[u]**; and

- a DFS root is an articulation point only when it has more than one DFS child.

Why parent edge ID matters

In a multigraph, two parallel edges may connect **u** and **v**:

TEXT

```
u ===== v
  edge 21
  edge 22
```

The DFS enters **v** through edge 21. It must skip that exact edge on the way back, but edge 22 is a legitimate back edge. Skipping every edge whose neighbor equals the parent would incorrectly label edge 21 a bridge.

The companion assigns and preserves edge IDs, carries `parentEdgeId`, and skips only that ID. It also rejects duplicate IDs because ambiguous identity would corrupt the result contract.

Numeric and representation policy

Graph code should state these policies before the algorithm:

- vertices are dense integers in $[0, V)$;
- missing adjacency is represented by absence, not a magic weight;
- weight magnitude is bounded from **V** so a vertex-length simple path stays strictly between **-INFINITY** and **INFINITY**;
- distance arithmetic uses `long` and guarded addition;
- directed and undirected edge records are separate types;
- undirected edge IDs are unique when identity matters; and
- recursion is educational unless depth constraints make it safe.

External IDs should be mapped to dense indexes at the boundary. Do not allocate a billion-slot array merely because one customer ID is large.

Edge-case matrix

Case	Correct handling	Frequent failure
DAG contains a cycle	reject the precondition	process a partial order as complete
negative DAG edge	valid	unnecessarily rejecting it like Dijkstra
unreachable negative cycle	Bellman-Ford source result remains valid	reporting every graph cycle as source-relevant
descendant of reachable negative cycle	mark undefined too	marking only cycle vertices
unreachable Floyd pair	keep INFINITY ; do not add it	overflow creates a fake negative path
multiple directed edges	initialize with minimum edge	last input edge wins arbitrarily
disconnected Prim input	return/report non-spanning result	calling a component tree an MST
negative MST edge	valid	importing Dijkstra's restriction
isolated SCC vertex	one singleton SCC	omitting vertices with no edges
undirected parallel edges	skip parent edge ID only	false bridge from parent-vertex skip
DFS root articulation	requires more than one DFS child	applying nonroot rule to root
self-loop	state semantics; not a bridge	treating all visited-neighbor edges identically
deep DFS chain	iterative frames or explicit limit	Java <code>StackOverflowError</code>
invalid endpoint	reject before indexing	accidental <code>ArrayIndexOutOfBoundsException</code>

Real interview follow-up round

Interviewer: Why can DAG shortest path handle a negative edge while Dijkstra cannot?

Candidate: Topological order guarantees every predecessor is processed before a vertex and no edge returns to a finalized prefix. Dijkstra instead relies on nonnegative weights to prove the smallest tentative distance cannot later decrease.

Interviewer: Bellman-Ford found a negative cycle in the graph. Are all source distances invalid?

Candidate: Only if the cycle is reachable from this source, and only vertices reachable from that cycle have undefined shortest distance. Unreachable vertices remain unreachable, while reachable vertices outside the cycle's downstream region can retain finite answers.

Interviewer: Why not use `Long.MAX_VALUE` as infinity?

Candidate: Adding any positive weight overflows. I use a separated sentinel, skip unreachable additions, constrain supported weights, and guard finite addition. The numeric policy is part of the API, not an implementation afterthought.

Interviewer: Prim and Dijkstra both use a priority queue. What differs?

Candidate: Dijkstra prioritizes total source-to-vertex distance and builds shortest paths. Prim prioritizes the weight of a crossing edge and minimizes total tree weight. Their queue syntax looks similar, but their invariants and outputs are different.

Interviewer: Why does the bridge condition use `>` but articulation uses `>=`?

Candidate: If a child's subtree can reach the parent itself, `low[child] == discovery[parent]`, the tree edge has an alternate route and is not a bridge. But removing the parent still separates that child subtree from the parent's ancestors, so the nonroot parent is an articulation point under equality.

Interviewer: How would you validate shortest-path implementations?

Candidate: On random DAGs, I can compare topological relaxation and Bellman-Ford from every source with Floyd-Warshall's corresponding row, including negative edges but no cycles. I also need directed fixed cases for reachable and unreachable negative cycles, plus infinity boundaries. The companion runs that differential test with a deterministic seed.

Run the verified companion

BASH

```
javac -Xlint:all -Werror GraphInterviewChecks.java
java GraphInterviewChecks
```

Expected final line:

TEXT

PASS 16 graph checks

For an interview, implement only the algorithm the contract needs, but be ready to defend why its invariant applies. A shorter correct choice beats a memorized advanced algorithm attached to the wrong graph.

SDE-2 CORE CHAPTER

Chapter 6 - Union-Find and Incremental Connectivity

Why this chapter exists

The foundations chapter named Union-Find and deferred it. This chapter develops it, because it is the one graph structure that answers a question BFS and DFS cannot answer efficiently: **connectivity that changes over time**.

BFS answers "are **a** and **b** connected?" in $O(V + E)$ per query. If edges arrive one at a time and you must answer after each arrival, that becomes $O(Q * (V + E))$ - quadratic and usually too slow. Union-Find answers each query in effectively constant time, and the reason it can is worth understanding rather than memorizing.

It also carries a specific interview signal. Candidates who reach for Union-Find on "number of provinces" when a DFS would do are pattern-matching rather than reasoning; candidates who cannot reach for it on "redundant connection" or "accounts merge" are missing a tool. Knowing which problems genuinely need it is the point.

WHY IT WORKS | First-principles model

Union-Find maintains a **partition of elements into disjoint sets** under two operations:

TEXT

```
find(x)      -> the representative of x's set
union(a, b)  -> merge the two sets containing a and b
```

Connectivity is then `find(a) == find(b)`. That is the whole interface.

The representation is a forest. Each element points at a parent; a root points at itself and is the set's representative. `find` walks to the root. `union` attaches one root to the other.

TEXT

```
initial:  0  1  2  3  4      every element its own root

union(0,1): 0      2  3  4
             |
             1

union(2,3):  0      2  4
             |      |
             1      3

union(1,3): find(1)=0, find(3)=2, attach 2 under 0

           0
          / \
         1  2
           |
           3
```

Naively this degenerates. Attaching roots arbitrarily can build a chain of length n , making `find` $O(n)$ and the whole structure no better than a linked list. Two independent optimizations fix that, and both are needed to reach the stated bound.

Specification boundary: the near-constant bound is *amortized*, not worst case. A single `find` can still be $O(\log n)$ before compression flattens the path. Union-Find also supports no efficient *undo* - it is an incremental structure, and removing an edge requires either rebuilding or a substantially more complex design. If a prompt deletes edges, Union-Find is the wrong tool and saying so is the correct answer.

The two optimizations

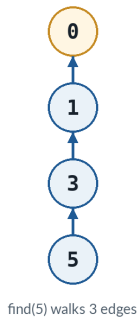
Union by size or rank. Always attach the smaller tree under the larger root. This alone bounds depth at $O(\log n)$, because an element's depth only increases when its tree is merged into one at least as large, which can happen at most $\log n$ times.

Path compression. During `find`, repoint every node on the path directly at the root. Subsequent finds on those nodes are $O(1)$.

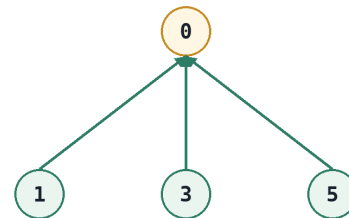
Path compression flattens the forest

`find(5)` repoints every node on the path directly at the root

before `find(5)`



after `find(5)`



Union by size alone: $O(\log n)$. Compression alone: $O(\log n)$. Both together: amortized $O(\alpha(n))$.

Path compression repoints every node on the find path directly at the root

Applied together, the amortized cost per operation is $O(\alpha(n))$, where α is the inverse Ackermann function. For every n that fits in the observable universe, $\alpha(n) < 5$. It is constant in practice and it is worth saying "effectively constant, formally inverse Ackermann" rather than claiming $O(1)$ outright - interviewers notice the difference.

Either optimization alone gives $O(\log n)$. Both give $O(\alpha(n))$. Knowing that they are independent, and that the bound requires both, is a standard follow-up.

Implementation

JAVA (continued 1/2)

```
public final class UnionFind {
    private final int[] parent;
    private final int[] size;
    private int componentCount;

    public UnionFind(int elements) {
        if (elements < 0) {
            throw new IllegalArgumentException("element count must be nonnegative");
        }
        parent = new int[elements];
        size = new int[elements];
        for (int i = 0; i < elements; i++) {
            parent[i] = i;          // every element starts as its own root
            size[i] = 1;
        }
        componentCount = elements;
    }

    /** Representative of x's set, compressing the path on the way out. */
    public int find(int x) {
        int root = x;
        while (parent[root] != root) {
            root = parent[root];
        }
        while (parent[x] != root) { // second pass repoints the whole path
            int next = parent[x];
            parent[x] = root;
            x = next;
        }
        return root;
    }
}
```

JAVA (continued 2/2)

```

/** Merge two sets. Returns false when they were already joined. */
public boolean union(int a, int b) {
    int rootA = find(a);
    int rootB = find(b);
    if (rootA == rootB) {
        return false;           // the edge is redundant
    }
    if (size[rootA] < size[rootB]) {
        int swap = rootA;
        rootA = rootB;
        rootB = swap;           // rootA is now the larger tree
    }
    parent[rootB] = rootA;
    size[rootA] += size[rootB];
    componentCount--;
    return true;
}

public boolean connected(int a, int b) {
    return find(a) == find(b);
}

public int componentCount() {
    return componentCount;
}

/** Size of the component containing x. */
public int componentSize(int x) {
    return size[find(x)];
}
}

```

Three details carry most of the value:

union returns a boolean. Whether the two elements were *already* connected is the answer to several problems by itself - redundant connection, cycle detection, and Kruskal's edge-acceptance test all hinge on it. Returning **void** throws that information away.

The iterative two-pass find. The recursive one-liner is shorter but recurses to the depth of the tree, which can be $O(n)$ before the first compression and will overflow the stack on a large adversarial input. The iterative version has no such failure mode.

size is only meaningful at a root. After a union, `size[rootB]` is stale. `componentSize` therefore calls `find` first. Reading `size[x]` directly for a non-root is a silent wrong answer, and it is a common bug.

What it solves, and what it does not

Problem	Union-Find?	Why
Count connected components, static graph	Either	DFS is simpler; use it unless edges stream in
Connectivity queries interleaved with edge additions	Yes	The case DFS cannot do efficiently
Redundant connection - find the edge closing a cycle	Yes	union returning false names the edge
Kruskal's minimum spanning tree	Yes	Accept an edge exactly when it joins two components
Accounts merge / entity resolution	Yes	Repeated merging by shared keys
Number of islands, single pass	Either	DFS or BFS is usually clearer
Number of islands II - islands appear one at a time	Yes	Incremental, which is the whole point
Detect a cycle in a directed graph	No	Union-Find has no notion of direction; use DFS colours or Kahn
Shortest path	No	It knows connectivity, not distance
Anything that removes edges	No	Not efficiently undoable

The directed-cycle row is the one interviewers probe. Union-Find treats every edge as symmetric, so it cannot distinguish $a \rightarrow b$ from $b \rightarrow a$ and cannot detect a directed cycle. Reaching for it there is a correctness error, not a performance one.

Kruskal's minimum spanning tree

Union-Find makes Kruskal almost trivial, which is why the two are usually taught together.

JAVA

```
record Edge(int from, int to, int weight) {}

static List<Edge> minimumSpanningTree(int vertices, List<Edge> edges) {
    List<Edge> sorted = new ArrayList<>(edges);
    sorted.sort(Comparator.comparingInt(Edge::weight));

    UnionFind sets = new UnionFind(vertices);
    List<Edge> chosen = new ArrayList<>();
    for (Edge edge : sorted) {
        if (sets.union(edge.from(), edge.to())) { // joins two components
            chosen.add(edge);
            if (chosen.size() == vertices - 1) {
                break; // a tree is complete
            }
        }
    }
    return chosen;
}
```

The correctness argument is the cut property: the lightest edge crossing any cut belongs to some minimum spanning tree. Processing edges in weight order and accepting exactly those that join two components realizes that greedily.

Complexity is $O(E \log E)$ for the sort, which dominates the $O(E * \alpha(V))$ of the union operations. **The sort is the bottleneck, not the Union-Find** - worth stating, because candidates often quote the alpha term as though it mattered.

If the graph is disconnected, `chosen` ends with fewer than $V - 1$ edges and you have a spanning *forest*. Whether that is an answer or an error is a requirement to clarify, not to assume.

Kruskal versus Prim

	Kruskal	Prim
Structure	Union-Find over sorted edges	Priority queue over frontier
Complexity	$O(E \log E)$	$O(E \log V)$ with a binary heap
Best when	Sparse graphs, edges already sorted	Dense graphs, adjacency matrix
Produces	Forest on a disconnected graph	Tree of one component only

The disconnected-graph difference is the practical one. Prim explores from a start vertex and silently returns a tree covering only that component; Kruskal covers everything it can. If the prompt does not promise connectivity, that distinction decides which is correct.

TRACE IT | Worked example: redundant connection

Given a graph that began as a tree and had exactly one extra edge added, return that edge.

The extra edge is precisely the first one whose endpoints are already connected.

JAVA

```
static int[] findRedundantConnection(int[][] edges) {
    UnionFind sets = new UnionFind(edges.length + 1); // 1-indexed vertices
    for (int[] edge : edges) {
        if (!sets.union(edge[0], edge[1])) {
            return edge; // already connected
        }
    }
    throw new IllegalArgumentException("no redundant edge; input was a tree");
}
```

The entire solution is the boolean that `union` returns. This is why the interface choice earlier mattered: with a `void` union you would need a separate `connected` call before every merge, which is the same work written less clearly.

Note the `edges.length + 1` sizing. The problem states vertices are labelled `1..n` and a tree with one extra edge has exactly `n` edges, so `n` vertices need indices `0..n` when 1-indexed. Off-by-one here produces an `ArrayIndexOutOfBoundsException` on the last vertex, and it is the most common bug in this problem.

WATCH OUT | Edge cases and common mistakes

- Using Union-Find to detect a cycle in a **directed** graph. It cannot; it has no direction.
- Attempting edge deletion. The structure is incremental and not efficiently undoable.
- Omitting union by size, so a chain forms and `find` degrades to $O(n)$.
- Omitting path compression, leaving $O(\log n)$ rather than $O(\alpha(n))$.
- Recursive `find` on a deep tree, overflowing the stack before compression can help.
- Reading `size[x]` without calling `find(x)` first; the value is only current at a root.
- Returning `void` from `union`, discarding the already-connected answer several problems depend on.
- Claiming $O(1)$ rather than amortized $O(\alpha(n))$. The distinction is small and interviewers notice.
- Off-by-one sizing for 1-indexed vertex labels.
- Quoting $\alpha(n)$ as Kruskal's bottleneck when the edge sort dominates at $O(E \log E)$.
- Assuming Kruskal returns a spanning tree on a disconnected graph. It returns a forest.
- Using Union-Find on a static connectivity count where a single DFS pass is simpler and equally fast.

TRY IT | Interview questions and model answers

What does Union-Find give you that BFS does not?

Efficient connectivity under *incremental* edge additions. BFS answers a connectivity query in $O(V + E)$, so interleaving Q queries with edge insertions costs $O(Q * (V + E))$. Union-Find answers each in amortized $O(\alpha(n))$. On a static graph where you just need components once, DFS is simpler and I would use it.

Why is the complexity $\alpha(n)$ and what are the two optimizations?

Union by size keeps trees shallow by attaching the smaller under the larger, giving $O(\log n)$ alone. Path compression flattens the path to the root during **find**, also giving $O(\log n)$ alone. Together they give amortized $O(\alpha(n))$, inverse Ackermann, which is under 5 for any practical n . Both are required for the combined bound.

Can you detect a cycle in a directed graph with Union-Find?

No. It models undirected connectivity and has no notion of edge direction, so it cannot distinguish $a \rightarrow b$ from $b \rightarrow a$. For directed cycles use DFS with white/grey/black colouring, or Kahn's algorithm and check whether the topological order covers every vertex.

Implement Kruskal and state the bottleneck.

Sort edges by weight, then accept an edge exactly when **union** reports it joined two distinct components, stopping at $V - 1$ edges. The bottleneck is the sort at $O(E \log E)$; the union operations are $O(E * \alpha(V))$ and are not the dominant term. On a disconnected graph this yields a spanning forest, which may or may not be the intended answer.

Your Union-Find is slow on a large input. What would you check?

Whether both optimizations are present - a missing union-by-size lets a chain form, and a missing path compression leaves logarithmic finds. Then whether **find** is recursive, which risks stack depth on a tall tree. Then whether component sizes are being read at non-roots, which is a correctness bug rather than a performance one.

When would you refuse Union-Find?

When edges are removed as well as added, since the structure is incremental with no efficient undo. When the question needs distances rather than connectivity. And when the graph is directed and the question is about reachability or cycles.

TRY IT | Exercises

- 1 **Foundation:** Draw the forest after `union(0,1)`, `union(2,3)`, `union(1,2)` with union by size, then show the effect of `find(3)` with path compression.
- 2 **Foundation:** Implement Union-Find without union by size and construct an input where `find` degrades to $O(n)$.
- 3 **Interview Core:** Implement the structure above with both optimizations, then count total pointer traversals across a million operations with and without compression.
- 4 **Interview Core:** Solve redundant connection. Then change the vertex labelling to 0-indexed and fix the sizing.
- 5 **Interview Core:** Implement Kruskal returning the chosen edges. Run it on a disconnected graph and state what you got back.
- 6 **Interview Core:** Solve "number of provinces" twice - once with DFS, once with Union-Find - and argue which you would present first in an interview.
- 7 **SDE-2 Follow-up:** Solve accounts merge, where accounts sharing any email belong to one person. Explain the mapping from emails to element indices.
- 8 **SDE-2 Follow-up:** Attempt directed cycle detection with Union-Find and construct the graph where it gives the wrong answer.
- 9 **SDE-2 Follow-up:** Extend the structure to report the size of the largest component in $O(1)$ after every union.
- 10 **Challenge:** Islands II - land cells appear one at a time on a grid; after each addition report the island count. Explain why an incremental structure is required.

RECAP | Chapter summary

Union-Find maintains a partition under `find` and `union`, and it exists to answer connectivity that changes as edges arrive - the case where repeated BFS becomes quadratic. Union by size and path compression are independent, each giving $O(\log n)$ alone and together giving amortized $O(\alpha(n))$, which is effectively but not literally constant. Have `union` return whether it actually merged: that boolean is the entire answer to redundant connection, undirected cycle detection, and Kruskal's acceptance test. Kruskal is then a short function whose bottleneck is the edge sort, not the union operations, and which yields a forest rather than a tree on a disconnected graph. The structure's limits are as important as its strengths: it has no direction, so it cannot detect directed cycles; it has no distances; and it has no efficient undo, so any prompt that deletes edges is telling you to use something else.

TRY IT | Revision checklist

- ☐ I can state the two operations and how connectivity is tested.
- ☐ I can explain why the naive forest degenerates and what each optimization fixes.
- ☐ I know both are needed for $O(\alpha(n))$ and that either alone gives $O(\log n)$.
- ☐ I implement `find` iteratively and can say why.
- ☐ My `union` returns whether a merge happened, and I know three problems that need it.
- ☐ I read component size only after `find`.
- ☐ I can implement Kruskal and name the sort as the bottleneck.
- ☐ I know Kruskal yields a forest on a disconnected graph and Prim does not.
- ☐ I can explain why Union-Find cannot detect a directed cycle.
- ☐ I recognize edge deletion as a signal to use a different structure.

SDE-2 CORE CHAPTER

Chapter 7 - Implicit Graphs and State-Space Search

Why this chapter exists

The hardest graph interview questions usually do not mention graphs. Word ladder, open the lock, minimum genetic mutation, sliding puzzle, jump game with teleports, the water-jug problem, minimum knight moves - none of these hands you a vertex list. Each is a graph where **the vertices are configurations and the edges are legal moves**, and the entire difficulty is seeing that.

Once you see it, the algorithm is one you already know. BFS finds the fewest moves. Dijkstra finds the cheapest when moves have different costs. The work moves from choosing an algorithm to **modelling the state and controlling its size**.

Candidates who have practised traversal on adjacency lists but never on implicit graphs tend to freeze on these, then write a bespoke recursive search that is exponential and wrong. Recognizing the shape is the skill.

WHY IT WORKS | First-principles model

An implicit graph is defined by a function rather than a data structure:

TEXT

explicit graph:	adj[u] is stored	-> read the neighbours
implicit graph:	neighbours(u) computed	-> generate the neighbours

Everything else about traversal is unchanged. The universal BFS invariant from chapter 1 still holds: **when a state is dequeued, its recorded distance is final**, provided every edge has the same cost.

An implicit graph has no edge list

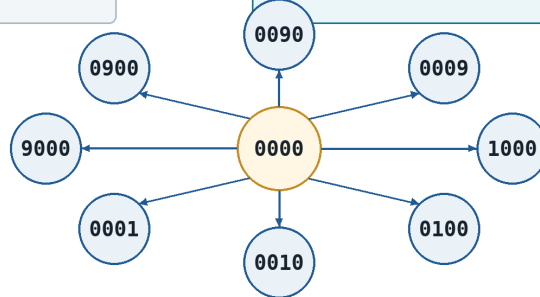
Vertices are configurations; edges are whatever the rules permit

explicit graph

```
adj[0] = [1, 2]
adj[1] = [3]
adj[2] = [3]
stored, then read
```

implicit graph

```
neighbours("0000") =
  turn one dial, either way
  -> 8 codes
computed on demand, never stored
```



State must exclude the path and the step count, and include everything a move depends on.

Java SDE-2 DSA Series | Volume 15: Graphs | Figure 02

An implicit graph generates neighbours instead of storing an edge list

Four questions replace the usual "build the adjacency list" step:

- 1 **What is a state?** The minimal information that determines the future. Not the path taken - only what constrains what happens next.
- 2 **What is the start, and what is the goal?** A goal may be a single state or a predicate.
- 3 **What are the legal moves?** The neighbour function.
- 4 **How large is the state space?** This is the feasibility check, and skipping it is the most common failure.

That fourth question is the one to answer out loud. $|V|$ for a 4-dial lock is 10^4 ; for word ladder it is the dictionary size; for an 8-puzzle it is $9!/2 = 181,440$; for a 15-puzzle it is about 10^{13} , which is why the 15-puzzle needs A* and heuristics rather than plain BFS.

Specification boundary: nothing here is a new algorithm. It is BFS, Dijkstra, and 0-1 BFS applied to a graph you generate rather than store. If your state space is too large to enumerate, no amount of BFS tuning saves you - the answer is a better state encoding, a heuristic search, or a different formulation.

Designing the state

Getting the state wrong is the expensive error, and it fails in two directions.

Too much state blows up the space. Recording the full path taken multiplies the state count by the number of paths, which is usually exponential. If two different routes arrive at the same configuration, they must be the *same* state - that collapsing is what makes BFS polynomial instead of exponential.

Too little state makes the answer wrong. If a move's legality depends on something you did not record, BFS will happily revisit a configuration in a context where the move is no longer legal.

TEXT

Word ladder	state = current word	(path not included)
Open the lock	state = 4-digit string	(turn count not included)
Sliding puzzle	state = board configuration	(moves so far not included)
Keys and doors	state = (position, bitmask of keys held)	(keys DO matter)
Bus routes	state = current route or stop	(choose carefully)

The keys-and-doors row is the instructive one. Position alone is insufficient because standing at a door with a key differs from standing at it without one. The key set is part of the state, and with $k \leq 6$ keys a bitmask makes $|V| = \text{rows} * \text{cols} * 2^k$, which is enumerable. This is the bitmask technique from the DP volume appearing as a graph state.

Distance is never part of the state. BFS tracks it separately. Folding it in makes every state unique and defeats the visited set entirely - a subtle bug that turns a linear search exponential while still producing correct answers on small inputs.

The visited set is a correctness control, not an optimization

In an explicit graph, forgetting **visited** costs time. In an implicit graph with cycles, it costs termination.

Mark visited when enqueueing, not when dequeuing. Marking on dequeue lets the same state enter the queue several times before it is first processed, which inflates the queue and can be quadratic.

JAVA

```
// Correct: the state can never be enqueued twice.
if (visited.add(next)) {
    queue.add(next);
}
```

Set.add returning a boolean makes this a single operation, which is both faster and harder to get wrong than a separate **contains** check followed by an **add**.

The set's element type matters more than it looks. States are often strings or arrays; **arrays do not implement value equality**, so a `HashSet<int[]>` silently never matches and the search never terminates. Encode to a `String`, a `record`, or a packed `long` - anything with meaningful `equals` and `hashCode`.

TRACE IT | Worked example: open the lock

A 4-dial lock starts at "0000". Each move turns one dial one step in either direction, with wraparound. Some codes are dead ends. Return the fewest moves to reach a target, or -1.

The four questions: a state is the 4-character code; start is "0000"; the goal is the target string; moves are the eight one-dial turns; the space is $10^4 = 10,000$ states with 8 edges each - trivially enumerable, so BFS is right.

JAVA (continued 1/2)

```
import java.util.*;

public final class OpenTheLock {

    static int openLock(String[] deadends, String target) {
        Set<String> blocked = new HashSet<>(Arrays.asList(deadends));
        if (blocked.contains("0000")) {
            return -1; // cannot even start
        }
        if (target.equals("0000")) {
            return 0; // check before the search, not inside it
        }

        Set<String> visited = new HashSet<>();
        Deque<String> queue = new ArrayDeque<>();
        queue.add("0000");
        visited.add("0000");
        int turns = 0;

        while (!queue.isEmpty()) {
            turns++; // one full level = one more move
            for (int level = queue.size(); level > 0; level--) {
                String code = queue.poll();
                for (String next : neighbours(code)) {
                    if (blocked.contains(next) || !visited.add(next)) {
                        continue;
                    }
                    if (next.equals(target)) {
                        return turns;
                    }
                    queue.add(next);
                }
            }
        }
    }
}
```


JAVA (continued 2/2)

```

    }
    }
}
return -1;
}

/** The eight codes reachable by turning one dial one step. */
private static List<String> neighbours(String code) {
    List<String> out = new ArrayList<>(8);
    char[] digits = code.toCharArray();
    for (int i = 0; i < 4; i++) {
        char original = digits[i];
        digits[i] = original == '9' ? '0' : (char) (original + 1);
        out.add(new String(digits));
        digits[i] = original == '0' ? '9' : (char) (original - 1);
        out.add(new String(digits));
        digits[i] = original;          // restore before the next dial
    }
    return out;
}

public static void main(String[] args) {
    System.out.println(openLock(
        new String[]{"0201", "0101", "0102", "1212", "2002"}, "0202")); // 6
    System.out.println(openLock(new String[]{"8888"}, "0009"));        // 1
    System.out.println(openLock(
        new String[]{"8887", "8889", "8878", "8898", "8788", "8988", "7888", "9888"},
        "8888")); // -1
    System.out.println(openLock(new String[]{"0000"}, "8888"));        // -1
}
}

```

Four details worth naming:

The level-by-level loop. Capturing `queue.size()` before the inner loop processes exactly one BFS level, so `turns` is the distance. Incrementing per *state* instead of per level is a classic wrong answer.

Restoring `digits[i]` before moving to the next dial. Without it, changes accumulate and the neighbour function generates codes two or three dials away - a bug that produces plausible but too-small answers.

Checking the target on generation rather than on dequeue returns one level earlier. Both are correct; this one is slightly faster and reads well.

The `target.equals("0000")` guard. Zero-move cases are the standard edge case, and handling them before the loop is cleaner than special-casing inside it.

Bidirectional BFS

When both start and goal are known and the branching factor b is uniform, searching from both ends and meeting in the middle reduces the frontier from $O(b^d)$ to $O(b^{(d/2)})$.

TEXT

one-directional:	b^d	e.g. $8^6 = 262,144$
bidirectional:	$2 * b^{(d/2)}$	e.g. $2 * 8^3 = 1,024$

The technique: keep two frontier sets, always expand the smaller one, and stop when a generated state appears in the other frontier.

It applies only when the goal is a *specific state* you can search backward from. A goal expressed as a predicate - "any configuration where all lights are off" - has no single backward start, and bidirectional search does not apply. Naming that condition is the interview signal; reciting the speedup is not.

For word ladder and open-the-lock it is a genuine and large improvement, and it is the standard follow-up once the plain BFS is working.

When the moves have costs

If every move costs the same, BFS is correct and $O(V + E)$. Once costs differ, the dequeue-is-final invariant breaks and you need the right tool:

Move costs	Algorithm	Note
All equal	BFS	Queue; distance is level number
Only 0 and 1	0-1 BFS	Deque; push-front on cost 0, push-back on cost 1
Arbitrary nonnegative	Dijkstra	Priority queue over states
Some negative	Bellman-Ford	Rare in state-space problems
Equal cost + good heuristic	A*	Needed once the space is too large to enumerate

0-1 BFS is worth remembering here because it appears naturally in grid problems where some moves are free - "minimum obstacles to remove", "minimum cost to make a path" - and it is $O(V + E)$ rather than Dijkstra's $O(E \log V)$ with no heap at all.

WATCH OUT | Edge cases and common mistakes

- Not recognizing the problem as a graph and writing a bespoke exponential recursion.
- Including the path or the move count in the state, so no two states ever collapse.
- Omitting information a move depends on - keys held, fuel remaining - so the search is wrong.
- Never computing the state-space size, then running BFS over something unenumerable.
- Using `HashSet<int[]>`; arrays have identity equality, so nothing ever matches and BFS never terminates.
- Marking visited on dequeue instead of on enqueue, inflating the queue.
- Incrementing the distance per state rather than per BFS level.
- Forgetting to restore mutated scratch state inside the neighbour function.
- Not handling start equal to goal, or a blocked start.
- Applying bidirectional BFS when the goal is a predicate rather than a specific state.
- Using BFS when move costs differ; the dequeue-is-final invariant no longer holds.
- Generating neighbours that leave the legal domain - off-grid, out-of-range, malformed - and relying on the visited set to absorb them.

TRY IT | Interview questions and model answers

How do you recognize a state-space graph problem?

The prompt asks for the fewest moves, steps, or transformations between configurations, and never supplies a vertex or edge list. Vertices are configurations, edges are legal moves, and the neighbour function replaces the adjacency list. Word ladder, open the lock, sliding puzzles, and minimum knight moves are all this shape.

How do you choose the state?

The minimal information that determines what may happen next. Not the path, since two routes reaching the same configuration must collapse to one state - that collapsing is what keeps BFS polynomial. But everything a move's legality depends on must be included, which is why keys-and-doors needs position plus a bitmask of keys held. Then compute the resulting space size to check the search is feasible at all.

Why must the visited set be marked on enqueue?

Marking on dequeue lets the same state be enqueued many times before it is first processed, inflating the queue and the work. Marking on enqueue guarantees each state enters once. `visited.add(next)` returning a boolean does the check and the mark in one operation.

Your BFS over board states never terminates. What would you check first?

The visited set's element type. If states are `int[]`, a `HashSet<int[]>` compares by identity, so no state is ever recognized as seen and cycles re-enter forever. Encode to a string, a record, or a packed long with real value equality.

When does bidirectional BFS help, and when does it not?

It helps when both endpoints are specific known states and the branching factor is roughly uniform, reducing the frontier from b^d to about $2 \cdot b^{(d/2)}$. It does not apply when the goal is a predicate rather than a single state, because there is nothing to search backward from.

The moves now have different costs. What changes?

BFS's invariant that a dequeued state has its final distance only holds for uniform costs. With costs of only 0 and 1, use 0-1 BFS on a deque - push-front for 0, push-back for 1 - which stays $O(V + E)$. With arbitrary nonnegative costs, use Dijkstra with a priority queue over states.

TRY IT | Exercises

- 1 **Foundation:** For word ladder, open the lock, and the 8-puzzle, write the state, start, goal, moves, and state-space size.
- 2 **Foundation:** Explain why including the path in the state makes BFS exponential, with a small concrete example.
- 3 **Interview Core:** Implement open the lock. Then remove the `digits[i] = original` restore line and describe the wrong answers produced.
- 4 **Interview Core:** Implement word ladder with a wildcard-bucket neighbour function, and state the complexity in terms of word length and dictionary size.
- 5 **Interview Core:** Solve the keys-and-doors grid with a `(position, keyMask)` state. Compute the state-space size for a 30x30 grid with 6 keys.
- 6 **Interview Core:** Store board states in a `HashSet<int[]>`, observe non-termination, then fix it with three different encodings and compare their speed.
- 7 **SDE-2 Follow-up:** Add bidirectional BFS to your open-the-lock solution and measure states expanded on a distance-6 target.
- 8 **SDE-2 Follow-up:** Solve "minimum obstacles to remove to reach the corner" with 0-1 BFS, then with Dijkstra, and compare runtimes.
- 9 **SDE-2 Follow-up:** Take a BFS that marks visited on dequeue and construct the input where the queue grows quadratically.
- 10 **Challenge:** Solve the 8-puzzle with BFS, report states expanded, then explain with numbers why the 15-puzzle needs A^* .

RECAP | Chapter summary

The hardest graph problems in interviews do not look like graph problems: vertices are configurations, edges are legal moves, and the adjacency list is replaced by a neighbour function you write. The algorithm is unchanged - BFS for uniform move costs, 0-1 BFS when costs are only zero and one, Dijkstra when they are arbitrary - so the work moves entirely into modelling. State must contain everything a move's legality depends on and nothing else: never the path or the step count, because two routes reaching one configuration have to collapse for the search to stay polynomial, and always the extras a move needs, such as a bitmask of keys held. Compute the state-space size before writing anything, because that number decides whether BFS is viable or whether the problem needs A*. The visited set is a termination control rather than an optimization: mark on enqueue, and make sure the element type has real value equality, since a `HashSet<int[]>` compares identities and lets a cyclic search run forever.

TRY IT | Revision checklist

- ☐ I recognize the implicit-graph shape from a "fewest moves" prompt with no edge list.
- ☐ I can answer the four modelling questions before writing code.
- ☐ I always compute the state-space size and use it to justify the algorithm.
- ☐ I exclude path and distance from state, and include everything a move depends on.
- ☐ I can explain why keys-and-doors needs a bitmask in the state.
- ☐ I mark visited on enqueue and know why dequeue is worse.
- ☐ I never put a raw array in a hash set, and can name three working encodings.
- ☐ I increment distance per BFS level, not per state.
- ☐ I can state when bidirectional BFS applies and when it cannot.
- ☐ I can pick between BFS, 0-1 BFS, and Dijkstra from the move-cost structure.

SDE-2 CORE CHAPTER

Chapter 8 - Graph Practice Lab

- 1 Model a dependency service as vertices and directed edges.
- 2 Compare adjacency list and matrix for sparse and dense graphs.
- 3 Explain enqueue-time visited marking in BFS.
- 4 Debug undirected cycle detection that treats the parent edge as a cycle.
- 5 Debug Dijkstra used on a graph containing a negative edge.
- 6 Count connected components in an undirected graph.
- 7 Reconstruct one shortest unweighted path with parent pointers.
- 8 Test whether a graph is bipartite across all components.
- 9 Implement 0-1 BFS.
- 10 Implement disjoint-set union with size and path compression.
- 11 Return edges of a minimum spanning tree and detect disconnection.
- 12 Compare topological sort with shortest path.
- 13 Choose an algorithm for dynamic edge additions and connectivity queries.
- 14 Explain recursion depth risk in graph DFS.
- 15 Design memory boundaries for a graph too large for one process.
- 16 **Interview Core:** Compute every grid cell's distance to the nearest source with one multi-source BFS. Explain the synthetic-source equivalence.
- 17 **SDE-2 Follow-up:** Give the shortest-path decision rule for unweighted, 0/1, nonnegative, negative-edge, DAG, and all-pairs inputs. Include reachable negative-cycle detection.

Advanced graph implementation lab

- 18 **Interview Core:** Implement DAG shortest paths with topological relaxation, negative edges, unreachable vertices, and explicit rejection of a cycle.
- 19 **Interview Core:** Implement Bellman-Ford returning finite distances plus an `affectedByNegativeCycle` flag for every vertex. Include reachable and unreachable negative cycles.
- 20 **Interview Core:** Implement Floyd-Warshall with a documented infinity sentinel, guarded addition, parallel-edge minimum, and negative-diagonal detection.
- 21 **Interview Core:** Implement Prim returning selected edge IDs, total `long` weight, and a non-spanning result for a disconnected graph.
- 22 **Interview Core:** Implement SCC decomposition and explain why the condensation graph is acyclic.
- 23 **SDE-2 Follow-up:** Return bridges and articulation points in an undirected multigraph. Carry parent edge ID so a parallel edge to the parent remains a back edge.
- 24 **SDE-2 Follow-up:** Differential-test DAG relaxation and Bellman-Ford from every source against Floyd-Warshall rows on randomly generated DAGs with signed weights.

SDE-2 CORE CHAPTER

Chapter 9 - Graph Practice Lab Solutions

- 1 A task/service step is a vertex; an edge points from prerequisite to dependent if topological processing should flow forward. State whether duplicate dependencies are meaningful.
- 2 Lists use $O(V + E)$ storage and traverse neighbors efficiently for sparse graphs. Matrices use $O(V^2)$, make edge checks $O(1)$, and may suit dense moderate graphs.
- 3 Marking when enqueued makes frontier membership equivalent to discovered state and prevents duplicate enqueues.
- 4 Carry the parent vertex in DFS/BFS. An already visited neighbor is a cycle only when it is not the edge back to the parent; parallel edges require an edge-identity policy.
- 5 Dijkstra's finalized-minimum argument fails with negative edges. Use Bellman-Ford, DAG relaxation, or another algorithm based on constraints.
- 6 Loop over every vertex; start a traversal from each unseen one and increment the count. Total $O(V + E)$.
- 7 BFS stores `parent[neighbor] = node` on first discovery. Walk backward from target and reverse. If undiscovered, no path exists.
- 8 Assign alternating colors during BFS/DFS. Check every component and fail on an edge whose endpoints share a color.
- 9 Use a deque. Relax weight-0 edges to the front and weight-1 edges to the back; stale-distance checks keep processing sound.
- 10 `find` compresses paths; `union` attaches smaller root under larger. Operations are near constant amortized and support connectivity, not paths.
- 11 Kruskal sorts edges and unions components; Prim grows from a start with a heap. A spanning tree requires exactly $V - 1$ accepted edges for nonempty connected input.
- 12 Topological sort is an ordering/cycle problem in DAGs. Shortest path optimizes cost; a DAG can use topological order as its evaluation schedule.
- 13 DSU handles undirected additions and connectivity efficiently. Deletions or directed reachability require different structures/offline techniques.
- 14 A chain can create $O(V)$ call depth and `StackOverflowError`; use an explicit deque for untrusted depth.
- 15 Partition vertices/edges, stream adjacency, use compressed IDs, external storage, frontier batching, and explicit consistency/failure contracts. Complexity alone does not solve distribution.
- 16 Enqueue all sources at distance zero and mark them discovered before traversal. Ordinary BFS then processes distance layers from the complete source set, equivalent to adding a synthetic source with zero-cost edges. Each traversable cell enters once, so time and output storage are $O(\text{rows} * \text{columns})$.

- 17 Use BFS for equal weights, deque-based 0-1 BFS for weights zero or one, Dijkstra for arbitrary nonnegative weights, topological relaxation for a DAG, Bellman-Ford when negative edges may exist, and Floyd-Warshall for suitable moderate all-pairs cases. Bellman-Ford relaxes all edges up to $V-1$ times; any further reachable improvement proves a reachable negative cycle.

Advanced graph implementation solutions

- 18 Compute a topological order and reject if it emits fewer than V vertices. Initialize source zero and others infinity; skip unreachable vertices and relax outgoing edges in order. Negative edges are valid because every predecessor appears before its destination. The companion's `dagShortestPaths` returns `[0, -1, 2, 1, INF]` on its signed-edge fixture and preserves the unreachable vertex.
- 19 Relax all edges up to $V-1$ passes, skipping unreachable `from` vertices and stopping after an unchanged pass. On one extra scan, seed every still-improvable reachable destination, then BFS/DFS outward to mark all vertices whose distances are undefined. Do not seed an edge from infinity. Return both numeric distances and flags because unreachable, finite, and negative-cycle-affected are three distinct states.
- 20 Initialize diagonal zero, other cells to a separated infinity sentinel, and parallel edges with their minimum weight. At phase `k`, combine only when both halves are reachable and use guarded `long` addition. A negative diagonal reports a negative cycle. Global cycle existence does not alone say every source-target pair is affected; pair-level invalidation also requires reachability through a negative-diagonal vertex.
- 21 Mark the start included and push its incident edges. Repeatedly remove the lightest edge; skip it when both endpoints share visited state, otherwise add the excluded endpoint, record the original edge, add weight in `long`, and push new crossing edges. Success requires exactly $V-1$ chosen edges for nonempty input. Returning edge IDs makes the result auditable; `spansAllVertices=false` distinguishes a component tree from an MST.
- 22 Kosaraju first records original-graph finish order, reverses edges, then explores in decreasing finish time to collect SCCs. If the condensation contained a directed cycle, all meta-vertices on that cycle would be mutually reachable and should have been one SCC, a contradiction. Thus condensation is a DAG. Include isolated vertices as singleton components.
- 23 Store `discovery` and `low`. DFS carries the exact incoming edge ID, not only parent vertex. Skip that one edge; a second parallel edge to the parent lowers `low` as a back edge. Add tree edge ID when `low[child] > discovery[parent]`. A nonroot parent is articulation when `low[child] >= discovery[parent]`; a root needs more than one DFS child. Reject duplicate IDs because result identity would be ambiguous.
- 24 Generate edges only from lower to higher indexes, guaranteeing a DAG while permitting signed weights. For every source, compare topological distances, Bellman-Ford distances, and the corresponding Floyd-Warshall row; assert no negative-cycle flag. Use a small signed range for duplicates and alternate routes, and a fixed seed for replay. This triangulation is stronger than comparing two algorithms that might share the same defect; fixed targeted cycle and infinity tests remain necessary.

SDE-2 CORE CHAPTER

Chapter 10 - Executable Graph Algorithm Checks

This dependency-free Java 21 companion validates traversal and ordering plus DAG shortest paths, Bellman-Ford affected states, guarded Floyd-Warshall, edge-returning Prim, SCCs, and edge-ID-correct bridges and articulation points. A successful run prints PASS 16 graph checks.

JAVA (continued 1/16)

```
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.Deque;
import java.util.HashSet;
import java.util.List;
import java.util.PriorityQueue;
import java.util.Random;
import java.util.Set;

public final class GraphInterviewChecks {
    static final long INFINITY = Long.MAX_VALUE / 4;

    record DirectedEdge(int from, int to, long weight) {}
    record UndirectedEdge(int id, int first, int second, long weight) {}
    record BellmanFordResult(long[] distances, boolean[] affectedByNegativeCycle) {
        boolean hasReachableNegativeCycle() {
            for (boolean affected : affectedByNegativeCycle) {
                if (affected) {
                    return true;
                }
            }
            return false;
        }
    }

    record FloydWarshallResult(long[][] distances, boolean hasNegativeCycle) {}
    record MstResult(List<UndirectedEdge> edges, long totalWeight, boolean spansAllVertices) {}
    record CriticalResult(Set<Integer> bridgeEdgeIds, Set<Integer> articulationVertices) {}

    private GraphInterviewChecks() {}

    static int[] courseOrder(int courses, int[][] prerequisites) {
        List<List<Integer>> graph = new ArrayList<>(courses);
        for (int i = 0; i < courses; i++) graph.add(new ArrayList<>());
        int[] indegree = new int[courses];
```

JAVA (continued 2/16)

```

    for (int[] pair : prerequisites) {
        graph.get(pair[1]).add(pair[0]);
        indegree[pair[0]]++;
    }
    Deque<Integer> ready = new ArrayDeque<>();
    for (int course = 0; course < courses; course++) {
        if (indegree[course] == 0) ready.addLast(course);
    }
    int[] order = new int[courses];
    int size = 0;
    while (!ready.isEmpty()) {
        int course = ready.removeFirst();
        order[size++] = course;
        for (int next : graph.get(course)) {
            if (--indegree[next] == 0) ready.addLast(next);
        }
    }
    return size == courses ? order : new int[0];
}

static int countIslands(char[][] grid) {
    int count = 0;
    int[][] directions = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
    for (int row = 0; row < grid.length; row++) {
        for (int column = 0; column < grid[row].length; column++) {
            if (grid[row][column] != '1') continue;
            count++;
            Deque<int[]> queue = new ArrayDeque<>();
            grid[row][column] = '0';
            queue.addLast(new int[] {row, column});
            while (!queue.isEmpty()) {
                int[] cell = queue.removeFirst();
                for (int[] direction : directions) {
                    int nextRow = cell[0] + direction[0];
                    int nextColumn = cell[1] + direction[1];
                    if (nextRow >= 0 && nextRow < grid.length && nextColumn >= 0

```

JAVA (continued 3/16)

```

        && nextColumn < grid[nextRow].length
        && grid[nextRow][nextColumn] == '1') {
    grid[nextRow][nextColumn] = '0';
    queue.addLast(new int[] {nextRow, nextColumn});
    }
    }
    }
    }
    return count;
}

/** Shortest paths in a DAG; negative edges are valid because no cycle exists. */
static long[] dagShortestPaths(int vertices, List<DirectedEdge> edges, int source) {
    requireVertex(source, vertices);
    List<List<DirectedEdge>> graph = directedGraph(vertices, edges);
    int[] indegree = new int[vertices];
    for (DirectedEdge edge : edges) {
        indegree[edge.to()]++;
    }
    Deque<Integer> ready = new ArrayDeque<>();
    for (int vertex = 0; vertex < vertices; vertex++) {
        if (indegree[vertex] == 0) {
            ready.addLast(vertex);
        }
    }
    int[] order = new int[vertices];
    int orderSize = 0;
    while (!ready.isEmpty()) {
        int vertex = ready.removeFirst();
        order[orderSize++] = vertex;
        for (DirectedEdge edge : graph.get(vertex)) {
            if (--indegree[edge.to()] == 0) {
                ready.addLast(edge.to());
            }
        }
    }
}

```

JAVA (continued 4/16)

```

    }
    if (orderSize != vertices) {
        throw new IllegalArgumentException("DAG shortest path requires an acyclic graph");
    }
    long[] distance = new long[vertices];
    Arrays.fill(distance, INFINITY);
    distance[source] = 0;
    for (int position = 0; position < order.length; position++) {
        int vertex = order[position];
        if (distance[vertex] == INFINITY) {
            continue;
        }
        for (DirectedEdge edge : graph.get(vertex)) {
            long candidate = finiteAdd(distance[vertex], edge.weight());
            if (candidate < distance[edge.to()]) {
                distance[edge.to()] = candidate;
            }
        }
    }
    return distance;
}

/**
 * Bellman-Ford distinguishes an unreachable negative cycle from one reachable
 * from the source and marks every vertex whose answer is then undefined.
 */
static BellmanFordResult bellmanFord(
    int vertices, List<DirectedEdge> edges, int source) {
    requireVertex(source, vertices);
    List<List<DirectedEdge>> graph = directedGraph(vertices, edges);
    long[] distance = new long[vertices];
    Arrays.fill(distance, INFINITY);
    distance[source] = 0;
    for (int pass = 1; pass < vertices; pass++) {
        long[] nextDistance = distance.clone();
        boolean changed = false;

```

JAVA (continued 5/16)

```
    for (DirectedEdge edge : edges) {
        if (distance[edge.from()] == INFINITY) {
            continue;
        }
        long candidate = finiteAdd(distance[edge.from()], edge.weight());
        if (candidate < nextDistance[edge.to()]) {
            nextDistance[edge.to()] = candidate;
            changed = true;
        }
    }
    distance = nextDistance;
    if (!changed) {
        break;
    }
}

boolean[] affected = new boolean[vertices];
Deque<Integer> queue = new ArrayDeque<>();
for (DirectedEdge edge : edges) {
    if (distance[edge.from()] != INFINITY
        && finiteAdd(distance[edge.from()], edge.weight()) < distance[edge.to()]
        && !affected[edge.to()]) {
        affected[edge.to()] = true;
        queue.addLast(edge.to());
    }
}
while (!queue.isEmpty()) {
    int vertex = queue.removeFirst();
    for (DirectedEdge edge : graph.get(vertex)) {
        if (!affected[edge.to()]) {
            affected[edge.to()] = true;
            queue.addLast(edge.to());
        }
    }
}
return new BellmanFordResult(distance, affected);
```

JAVA (continued 6/16)

```

    }

    /** All-pairs shortest paths with an explicit sentinel and saturated addition. */
    static FloydWarshallResult floydWarshall(
        int vertices, List<DirectedEdge> edges) {
        directedGraph(vertices, edges); // validates vertices and supported weights
        long[][] distance = new long[vertices][vertices];
        for (long[] row : distance) {
            Arrays.fill(row, INFINITY);
        }
        for (int vertex = 0; vertex < vertices; vertex++) {
            distance[vertex][vertex] = 0;
        }
        for (DirectedEdge edge : edges) {
            distance[edge.from()][edge.to()] =
                Math.min(distance[edge.from()][edge.to()], edge.weight());
        }
        for (int through = 0; through < vertices; through++) {
            for (int from = 0; from < vertices; from++) {
                if (distance[from][through] == INFINITY) {
                    continue;
                }
                for (int to = 0; to < vertices; to++) {
                    if (distance[through][to] == INFINITY) {
                        continue;
                    }
                    long candidate = finiteAdd(
                        distance[from][through], distance[through][to]);
                    if (candidate < distance[from][to]) {
                        distance[from][to] = candidate;
                    }
                }
            }
        }
        boolean negativeCycle = false;
        for (int vertex = 0; vertex < vertices; vertex++) {

```

JAVA (continued 7/16)

```

        negativeCycle |= distance[vertex][vertex] < 0;
    }
    return new FloydWarshallResult(distance, negativeCycle);
}

/** Prim returns the chosen original edges, not only the total weight. */
static MstResult primMst(
    int vertices, List<UndirectedEdge> edges, int start) {
    requireVertex(start, vertices);
    List<List<UndirectedEdge>> graph = undirectedGraph(vertices, edges);
    boolean[] visited = new boolean[vertices];
    PriorityQueue<UndirectedEdge> frontier = new PriorityQueue<>(
        Comparator.comparingLong(UndirectedEdge::weight)
            .thenComparingInt(UndirectedEdge::id));
    List<UndirectedEdge> chosen = new ArrayList<>();
    long total = 0;
    visited[start] = true;
    frontier.addAll(graph.get(start));
    while (!frontier.isEmpty() && chosen.size() < vertices - 1) {
        UndirectedEdge edge = frontier.remove();
        boolean firstVisited = visited[edge.first()];
        boolean secondVisited = visited[edge.second()];
        if (firstVisited == secondVisited) {
            continue;
        }
        int next = firstVisited ? edge.second() : edge.first();
        visited[next] = true;
        chosen.add(edge);
        total = finiteAdd(total, edge.weight());
        for (UndirectedEdge candidate : graph.get(next)) {
            int other = candidate.first() == next
                ? candidate.second() : candidate.first();
            if (!visited[other]) {
                frontier.add(candidate);
            }
        }
    }
}

```


JAVA (continued 8/16)

```

    }
    boolean spans = chosen.size() == Math.max(0, vertices - 1);
    return new MstResult(List.copyOf(chosen), total, spans);
}

/** Kosaraju returns one list per strongly connected component. */
static List<List<Integer>> stronglyConnectedComponents(
    int vertices, List<DirectedEdge> edges) {
    List<List<DirectedEdge>> graph = directedGraph(vertices, edges);
    List<List<Integer>> reverse = emptyAdjacency(vertices);
    for (DirectedEdge edge : edges) {
        reverse.get(edge.to()).add(edge.from());
    }
    boolean[] visited = new boolean[vertices];
    List<Integer> finishOrder = new ArrayList<>(vertices);
    for (int vertex = 0; vertex < vertices; vertex++) {
        if (!visited[vertex]) {
            finishDfs(vertex, graph, visited, finishOrder);
        }
    }
    Arrays.fill(visited, false);
    List<List<Integer>> components = new ArrayList<>();
    for (int position = finishOrder.size() - 1; position >= 0; position--) {
        int vertex = finishOrder.get(position);
        if (!visited[vertex]) {
            List<Integer> component = new ArrayList<>();
            collectDfs(vertex, reverse, visited, component);
            component.sort(Integer::compare);
            components.add(component);
        }
    }
    components.sort(Comparator.comparingInt(component -> component.get(0)));
    return components;
}

/** Tarjan low-link logic skips only the parent edge ID, so parallel edges work. */

```

JAVA (continued 9/16)

```

static CriticalResult bridgesAndArticulationPoints(
    int vertices, List<UndirectedEdge> edges) {
    List<List<AdjacentEdge>> graph = emptyAdjacentEdges(vertices);
    Set<Integer> ids = new HashSet<>();
    for (UndirectedEdge edge : edges) {
        requireVertex(edge.first(), vertices);
        requireVertex(edge.second(), vertices);
        if (!ids.add(edge.id())) {
            throw new IllegalArgumentException("edge IDs must be unique");
        }
        graph.get(edge.first()).add(new AdjacentEdge(edge.second(), edge.id()));
        graph.get(edge.second()).add(new AdjacentEdge(edge.first(), edge.id()));
    }
    int[] discovery = new int[vertices];
    int[] low = new int[vertices];
    Arrays.fill(discovery, -1);
    int[] clock = {0};
    Set<Integer> bridges = new HashSet<>();
    Set<Integer> articulation = new HashSet<>();
    for (int vertex = 0; vertex < vertices; vertex++) {
        if (discovery[vertex] == -1) {
            criticalDfs(vertex, -1, graph, discovery, low, clock,
                bridges, articulation);
        }
    }
    return new CriticalResult(Set.copyOf(bridges), Set.copyOf(articulation));
}

record AdjacentEdge(int to, int id) {}

private static void criticalDfs(
    int vertex,
    int parentEdgeId,
    List<List<AdjacentEdge>> graph,
    int[] discovery,
    int[] low,

```

JAVA (continued 10/16)

```

        int[] clock,
        Set<Integer> bridges,
        Set<Integer> articulation) {
    discovery[vertex] = low[vertex] = clock[0]++;
    int childCount = 0;
    for (AdjacentEdge edge : graph.get(vertex)) {
        if (edge.id() == parentEdgeId) {
            continue;
        }
        if (discovery[edge.to()] != -1) {
            low[vertex] = Math.min(low[vertex], discovery[edge.to()]);
            continue;
        }
        childCount++;
        criticalDfs(edge.to(), edge.id(), graph, discovery, low, clock,
            bridges, articulation);
        low[vertex] = Math.min(low[vertex], low[edge.to()]);
        if (low[edge.to()] > discovery[vertex]) {
            bridges.add(edge.id());
        }
        if (parentEdgeId != -1 && low[edge.to()] >= discovery[vertex]) {
            articulation.add(vertex);
        }
    }
    if (parentEdgeId == -1 && childCount > 1) {
        articulation.add(vertex);
    }
}

private static void finishDfs(
    int vertex,
    List<List<DirectedEdge>> graph,
    boolean[] visited,
    List<Integer> order) {
    visited[vertex] = true;
    for (DirectedEdge edge : graph.get(vertex)) {

```

JAVA (continued 11/16)

```
        if (!visited[edge.to()]) {
            finishDfs(edge.to(), graph, visited, order);
        }
    }
    order.add(vertex);
}

private static void collectDfs(
    int vertex,
    List<List<Integer>> graph,
    boolean[] visited,
    List<Integer> component) {
    visited[vertex] = true;
    component.add(vertex);
    for (int next : graph.get(vertex)) {
        if (!visited[next]) {
            collectDfs(next, graph, visited, component);
        }
    }
}

private static List<List<DirectedEdge>> directedGraph(
    int vertices, List<DirectedEdge> edges) {
    if (vertices < 0) {
        throw new IllegalArgumentException("vertex count cannot be negative");
    }
    List<List<DirectedEdge>> graph = new ArrayList<>(vertices);
    for (int vertex = 0; vertex < vertices; vertex++) {
        graph.add(new ArrayList<>());
    }
    for (DirectedEdge edge : edges) {
        requireVertex(edge.from(), vertices);
        requireVertex(edge.to(), vertices);
        requireSupportedWeight(edge.weight(), vertices);
        graph.get(edge.from()).add(edge);
    }
}
```

JAVA (continued 12/16)

```
        return graph;
    }

    private static List<List<UndirectedEdge>> undirectedGraph(
        int vertices, List<UndirectedEdge> edges) {
        if (vertices < 0) {
            throw new IllegalArgumentException("vertex count cannot be negative");
        }
        List<List<UndirectedEdge>> graph = new ArrayList<>(vertices);
        for (int vertex = 0; vertex < vertices; vertex++) {
            graph.add(new ArrayList<>());
        }
        for (UndirectedEdge edge : edges) {
            requireVertex(edge.first(), vertices);
            requireVertex(edge.second(), vertices);
            requireSupportedWeight(edge.weight(), vertices);
            graph.get(edge.first()).add(edge);
            graph.get(edge.second()).add(edge);
        }
        return graph;
    }

    private static List<List<Integer>> emptyAdjacency(int vertices) {
        List<List<Integer>> graph = new ArrayList<>(vertices);
        for (int vertex = 0; vertex < vertices; vertex++) {
            graph.add(new ArrayList<>());
        }
        return graph;
    }

    private static List<List<AdjacentEdge>> emptyAdjacentEdges(int vertices) {
        List<List<AdjacentEdge>> graph = new ArrayList<>(vertices);
        for (int vertex = 0; vertex < vertices; vertex++) {
            graph.add(new ArrayList<>());
        }
        return graph;
    }
```

JAVA (continued 13/16)

```

    }

    private static void requireVertex(int vertex, int vertices) {
        if (vertices < 0 || vertex < 0 || vertex >= vertices) {
            throw new IllegalArgumentException("vertex is outside [0,V)");
        }
    }

    private static void requireSupportedWeight(long weight, int vertices) {
        long pathLengthGuard = Math.max(1L, (long) vertices + 1L);
        long maximumMagnitude = (INFINITY - 1L) / pathLengthGuard;
        if (weight < -maximumMagnitude || weight > maximumMagnitude) {
            throw new IllegalArgumentException(
                "edge weight is too large for the graph's infinity policy");
        }
    }

    private static long finiteAdd(long first, long second) {
        if (first == INFINITY || second == INFINITY) {
            return INFINITY;
        }
        if (first == -INFINITY || second == -INFINITY) {
            return -INFINITY;
        }
        if (second > 0 && first > INFINITY - second) {
            return INFINITY;
        }
        if (second < 0 && first < -INFINITY - second) {
            return -INFINITY;
        }
        return first + second;
    }

    private static boolean shortestPathAlgorithmsAgreeOnRandomDags() {
        Random random = new Random(43L);
        for (int trial = 0; trial < 200; trial++) {

```

JAVA (continued 14/16)

```

    int vertices = 2 + random.nextInt(8);
    List<DirectedEdge> edges = new ArrayList<>();
    for (int from = 0; from < vertices; from++) {
        for (int to = from + 1; to < vertices; to++) {
            if (random.nextInt(4) == 0) {
                edges.add(new DirectedEdge(from, to, random.nextInt(21) - 10));
            }
        }
    }
    FloydWarshallResult allPairs = floydWarshall(vertices, edges);
    for (int source = 0; source < vertices; source++) {
        long[] dag = dagShortestPaths(vertices, edges, source);
        BellmanFordResult bellman = bellmanFord(vertices, edges, source);
        if (!Arrays.equals(dag, bellman.distances())
            || !Arrays.equals(dag, allPairs.distances()[source])
            || bellman.hasReachableNegativeCycle()) {
            return false;
        }
    }
    return true;
}

private static void check(boolean condition, String message) {
    if (!condition) throw new AssertionError(message);
}

public static void main(String[] args) {
    int[] order = courseOrder(4, new int[][] {{1, 0}, {2, 0}, {3, 1}, {3, 2}});
    check(order.length == 4 && order[0] == 0 && order[3] == 3, "course order");
    check(courseOrder(2, new int[][] {{0, 1}, {1, 0}}).length == 0, "cycle");
    char[][] grid = {{'1', '1', '0'}, {'0', '1', '0'}, {'1', '0', '1'}};
    check(countIslands(grid) == 3, "islands");
    check(Arrays.stream(order).distinct().count() == 4L, "unique order");
}

```

JAVA (continued 15/16)

```

List<DirectedEdge> dag = List.of(
    new DirectedEdge(0, 1, 4),
    new DirectedEdge(0, 2, 2),
    new DirectedEdge(2, 1, -3),
    new DirectedEdge(1, 3, 2),
    new DirectedEdge(2, 3, 5));
check(Arrays.equals(dagShortestPaths(5, dag, 0),
    new long[] {0, -1, 2, 1, INFINITY}), "DAG shortest paths");

BellmanFordResult normal = bellmanFord(5, dag, 0);
check(Arrays.equals(normal.distances(), new long[] {0, -1, 2, 1, INFINITY})
    && !normal.hasReachableNegativeCycle(), "Bellman-Ford finite answers");
List<DirectedEdge> reachableCycle = List.of(
    new DirectedEdge(0, 1, 1),
    new DirectedEdge(1, 2, -2),
    new DirectedEdge(2, 1, -2),
    new DirectedEdge(2, 3, 1));
BellmanFordResult cycleResult = bellmanFord(4, reachableCycle, 0);
check(cycleResult.hasReachableNegativeCycle()
    && cycleResult.affectedByNegativeCycle()[1]
    && cycleResult.affectedByNegativeCycle()[2]
    && cycleResult.affectedByNegativeCycle()[3]
    && !cycleResult.affectedByNegativeCycle()[0], "reachable negative cycle propagation");
BellmanFordResult unreachableCycle = bellmanFord(4, List.of(
    new DirectedEdge(0, 1, 1),
    new DirectedEdge(2, 3, -1),
    new DirectedEdge(3, 2, -1)), 0);
check(!unreachableCycle.hasReachableNegativeCycle(), "unreachable cycle is irrelevant");

FloydWarshallResult allPairs = floydWarshall(5, dag);
check(allPairs.distances()[0][3] == 1
    && allPairs.distances()[4][0] == INFINITY
    && !allPairs.hasNegativeCycle(), "Floyd-Warshall infinity policy");
check(floydWarshall(3, reachableCycle.subList(1, 3)).hasNegativeCycle(),
    "Floyd-Warshall negative-cycle diagonal");

```


JAVA (continued 16/16)

```

List<UndirectedEdge> weighted = List.of(
    new UndirectedEdge(10, 0, 1, 4),
    new UndirectedEdge(11, 0, 2, 1),
    new UndirectedEdge(12, 2, 1, 2),
    new UndirectedEdge(13, 1, 3, 1),
    new UndirectedEdge(14, 2, 3, 5));
MstResult mst = primMst(4, weighted, 0);
check(mst.spansAllVertices() && mst.totalWeight() == 4 && mst.edges().size() == 3,
    "Prim returns MST edges and weight");
check(!primMst(3, List.of(new UndirectedEdge(1, 0, 1, 2)), 0)
    .spansAllVertices(), "Prim reports disconnected graph");

List<List<Integer>> components = stronglyConnectedComponents(6, List.of(
    new DirectedEdge(0, 1, 1), new DirectedEdge(1, 0, 1),
    new DirectedEdge(1, 2, 1), new DirectedEdge(2, 3, 1),
    new DirectedEdge(3, 2, 1), new DirectedEdge(3, 4, 1)));
check(components.equals(List.of(List.of(0, 1), List.of(2, 3), List.of(4), List.of(5))),
    "strongly connected components");

CriticalResult critical = bridgesAndArticulationPoints(4, List.of(
    new UndirectedEdge(20, 0, 1, 0),
    new UndirectedEdge(21, 1, 2, 0),
    new UndirectedEdge(22, 1, 2, 0),
    new UndirectedEdge(23, 2, 3, 0)));
check(critical.bridgeEdgeIds().equals(Set.of(20, 23)),
    "parallel edge is not a bridge");
check(critical.articulationVertices().equals(Set.of(1, 2)),
    "articulation vertices");
check(shortestPathAlgorithmsAgreeOnRandomDags(),
    "random shortest-path differential test");

System.out.println("PASS 16 graph checks");
}
}

```

Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: traversal and components
- Interview Core: cycle detection, topological sort, and shortest paths
- SDE-2 Follow-up: path reconstruction and scale
- Challenge: adapt algorithms to grids or evolving connectivity

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

[Previous book: DSA 14 - Heaps, Priority Queues, Selection, and Top-K](#)

[Series index](#)

[Next book: DSA 16 - Greedy Algorithms: Recognition, Proofs, and Scheduling](#)

TRY IT | Completion check

- ☐ I can recognize the core patterns without being told the data structure or algorithm.
- ☐ I can implement the representative Java templates from memory.
- ☐ I can explain why each implementation is correct.
- ☐ I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- ☐ I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	JAVA 01	Java Foundations for Problem Solving
Java	JAVA 02	Git and GitHub for Java Engineers
Java	JAVA 03	Maven and Gradle for Java Engineers
Java	JAVA 04	Language, OOP, and Modern Java
Java	JAVA 05	Collections, Streams, and I/O
Java	JAVA 06	JVM and Java Execution
Java	JAVA 07	Java Concurrency and the Memory Model
Java	JAVA 08	Performance, Diagnostics, and GC Incidents
Java	JAVA 09	Java Question Bank, Study Plan, and Reference
DSA	DSA 01	Time and Space Complexity for Java Interviews
DSA	DSA 02	Number Systems and Math Foundations for DSA Interviews
DSA	DSA 03	Number Systems Interview Patterns and Rapid Revision
DSA	DSA 04	Bit Manipulation in Java
DSA	DSA 05	Loop Mastery, Patterns, and Index Calculations
DSA	DSA 06	Arrays and Array Problem-Solving Patterns
DSA	DSA 07	Strings and String Problem-Solving Patterns
DSA	DSA 08	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	DSA 09	Recursion and Backtracking in Java
DSA	DSA 10	Linked Lists: Pointer Reasoning and Mutation
DSA	DSA 11	Stacks, Queues, Deques, and Monotonic Patterns
DSA	DSA 12	Binary Search: Bounds, Answers, and Invariants
DSA	DSA 13	Trees, Binary Search Trees, and Tries
DSA	DSA 14	Heaps, Priority Queues, Selection, and Top-K
DSA	DSA 15	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	DSA 16	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	DSA 17	Dynamic Programming: State, Transitions, and Optimization
Frameworks	FW 01	MySQL for Java Backend Interviews
Frameworks	FW 02	Hibernate and JPA for Java Backend Interviews
Frameworks	FW 03	Spring Framework for Java Engineers
Frameworks	FW 04	Spring Boot for Java Backend Engineers
Frameworks	FW 05	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	FW 06	Spring Data for Java Backend Engineers
Frameworks	FW 07	MongoDB for Java Backend Interviews
Frameworks	FW 08	Redis for Java Backend Interviews
Frameworks	FW 09	Persistence, SQL, JPA, and Caching
Frameworks	FW 10	Apache Kafka and Spring Kafka
Frameworks	FW 11	Spring Ecosystem Extensions
Frameworks	FW 12	Spring AI for Java Engineers
System Design	SD 01	Design, Backend, Testing, and Security
System Design	SD 02	Distributed Systems and System Design
System Design	SD 03	Generative AI System Design

All 40 publication editions are available now. Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that shelf in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: [LinkedIn](#) | [GitHub](#)

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Data Structures and Algorithms - Book 15 of 17 - Study Step DSA-15. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.